UNIVERSITY OF CALIFORNIA

SANTA CRUZ

SATURN: ZERO-CONFIGURATION AI SERVICE DISCOVERY

A thesis submitted in partial satisfaction

of the requirements for the degree of

MASTER OF SCIENCE

in

COMPUTER SCIENCE

by

Joseph Perrello

March 2026

The Thesis of Joseph Perrello

is approved:

Adam Smith, Chair

Ram Sundara Raman

Peter Biehl

Vice Provost and Dean of Graduate Studies

Copyright © by

Joseph Perrello

2026

Contents

List of Figures **x**

List of Tables **xii**

Abstract **xiii**

Acknowledgments **xiv**

1 Introduction **1**

1.1 The Problem: AI Access as Configuration Burden 1

1.2 The Vision: AI as Network Infrastructure 2

1.3 Saturn in Brief 3

1.4 Thesis Statement 3

1.5 Contributions 4

1.6 Organization 4

2 Background **6**

2.1 The AI Service Landscape 6

2.1.1 The OpenAI-Compatible API 6

2.1.2 Configuration Fragmentation 7

2.1.3 Static API Key Fragility 7

2.2 mDNS and DNS-SD 8

2.2.1 Multicast DNS 9
2.2.2 DNS-Based Service Discovery 9
2.2.3 TXT Records and Metadata 9
2.2.4 Platform Adoption 10
2.3 Alternative Discovery Protocols 10
2.4 The DHCP Parallel 11
2.5 Security Context 12
2.5.1 Zero-Trust and BeyondCorp 12
2.5.2 mDNS Privacy and Vulnerability Surface 12
2.5.3 The Gap Saturn Addresses 13

3 Design **14**
3.1 Goals 14
3.2 Audiences 15
3.3 Design Origins 16
3.4 Concepts 17
3.4.1 Endpoints 17
3.4.2 Beacons 18
3.4.3 Priorities 18
3.4.4 Ephemeral Keys 19
3.5 Protocol Specification 20
3.5.1 Service Type 20
3.5.2 Record Structure 21
3.5.3 Endpoint Expectations 21
3.5.4 Discovery Process 22
3.6 Architecture Decisions 24

3.6.1 Beacon, Not Proxy 24
3.6.2 Multicast, Not Registry 24
3.6.3 OpenAI-Compatible API, Not a Custom Protocol 25
3.6.4 Protocol, Not Implementation 25

4 Implementation **26**
4.1 Core Python Package 27
4.1.1 Discovery Layer 27
4.1.2 Server Types 28
4.1.3 Configuration System 28
4.2 Embedded Deployment 29
4.2.1 GL.iNet Router Deployment 29
4.3 Client Integration Patterns 31
4.3.1 Native SDK Integration 32
4.3.2 Application Plugin 34
4.3.3 Tool Protocol Bridge 34
4.3.4 Subprocess Bridge 35
4.3.5 Application Fork 36
4.3.6 Localhost Proxy 37
4.3.7 Environment Variable Injection 38
4.4 System Walkthrough 38
4.4.1 Administrator: Deploying a Beacon 38
4.4.2 Developer: Discovering and Using a Service 39
4.4.3 End User: AI Without Configuration 39

5 Evaluation **41**
5.1 Evaluation Approach 41

5.1.1 Requirements Decomposition 42
5.2 Discovery Feasibility (Claim 1) 43
5.2.1 R1: Service Discovery 43
5.2.2 R2: Connection Sufficiency 43
5.2.3 R3: Cross-Implementation Interoperability 45
5.2.4 Integration Opportunity 46
5.3 Configuration Effort (Claim 2) 48
5.3.1 Methodology 48
5.3.2 Results 49
5.3.3 Analysis 50
5.4 Usability Heuristic Evaluation 51
5.4.1 Method 51
5.4.2 Strengths 52
5.4.3 Trade-offs 52
5.4.4 Limitations 53
5.4.5 Synthesis 54
5.5 Security Analysis (Claim 3) 54
5.5.1 Information Leakage Surface 55
5.5.2 STRIDE Threat Comparison 56
5.5.3 Exposure Window Analysis 56
5.5.4 AP Isolation and Deployment Constraints 57
5.5.5 Security Posture Across Implementations 58
5.6 Threats to Validity 58

6 Discussion **60**
6.1 Threat Model 60

6.1.1 Home Network 60
6.1.2 Campus Lab or Classroom 61
6.1.3 Coffee Shop 61
6.1.4 Compromised Administrator 62
6.2 The BeyondCorp Divergence 62
6.3 API Key Leakage: A Comparison 63
6.4 AP Isolation: The Hard Blocker 64
6.5 Limitations and Generalizability 64
6.6 Relation to Prior Work 66
6.6.1 Service Discovery 66
6.6.2 AI Provisioning 66
6.6.3 Network Security 67
6.7 Future Work 67
6.7.1 Privacy-Preserving Discovery 67
6.7.2 Tiered Trust 68
6.7.3 Wide-Area Discovery 68
6.8 Broader Implications 68

7 Conclusion **70**

A Saturn Protocol Specification **72**
A.1 Service Registration 72
A.2 TXT Record Schema 73
A.3 Endpoint Requirements 73
A.4 Discovery Algorithm 74
A.5 Ephemeral Key Rotation Protocol 75
A.6 Conformance Levels 76

A.6.1 Beacon Conformance 76
A.6.2 Client Conformance 77

B Generative AI in Thesis Development 80

B.1 Knowledge Graph Authorship 80
B.2 Iterative Advisory Role 81
B.3 Design Fictions as Specification Input 82
B.4 Failed Evaluation: A Cautionary Note 83
B.5 Limits and Human Judgment 84

C Statistical Details 85

C.1 Confidence Interval Computation 85
C.2 Claim 1: Discovery Latency Raw Data 86
C.3 Claim 2: Configuration Effort Raw Data 87
C.3.1 Cognitive Walkthrough Step Counts 88
C.4 Sample Size Analysis 89
C.4.1 Limitations of the Statistical Approach 89

D Scenario Specifications 91

D.1 Claim 1: Discovery (Cold Start) 91
D.1.1 Task Description 91
D.1.2 Environment 92
D.1.3 Measurements 93
D.1.4 Success Criteria 93
D.2 Claim 2: Configuration Effort 94
D.2.1 Control Condition: Manual Ollama Setup 94
D.2.2 Treatment Condition: Saturn-Enabled Setup 94

D.2.3 Environment 95
D.2.4 Measurements 96
D.2.5 Key Differences Between Conditions 96
D.2.6 Reproducibility 97

List of Figures

4.1 Terminal output from `saturn run ollama`. The beacon registers on mDNS and advertises the backend's models, endpoint, and metadata. . . 39

4.2 Output from `saturn discover`, confirming the service is visible via mDNS. 39

4.3 Python session demonstrating Saturn's two-function discovery API. The call resolves endpoint, authentication, and model list from mDNS at runtime. 40

List of Tables

3.1 Saturn TXT Record Schema 22

3.2 Required Saturn Endpoint Paths 23

4.1 Saturn Implementation Components 26

5.1 Connection concerns mapped to DNS-SD records. 44

5.2 API backends operating through Saturn's TXT record schema. 44

5.3 Saturn implementation matrix. Seven consumers span three languages, four mDNS libraries, and four integration patterns. 45

5.4 mDNS libraries used across Saturn implementations. 45

5.5 Integration targets by category and viability. "Endpoint" means the application already accepts custom OpenAI-compatible API URLs. "Plugin" means the application supports extensions that could perform Saturn discovery. "Proxy" means the application would require an external bridge (like VLC's subprocess pattern). 47

5.6 Sysadmin configuration steps. 49

5.7 App developer configuration steps. 49

5.8 End user additional steps (beyond shared app onboarding). 50

5.9 Configuration steps by persona (Saturn package vs. traditional). 51

5.10 Usability heuristics where Saturn demonstrates strength. 52

5.11 Usability heuristics where Saturn presents trade-offs. 53

5.12 Usability heuristics where Saturn shows limitations. 54
5.13 Fields exposed by Saturn protocol design. 55
5.14 STRIDE severity by credential approach. 56
5.15 Credential exposure window comparison. 57

A.1 Complete TXT Record Field Reference 79

C.1 Per-Trial Discovery Latency (ms) 86
C.2 Per-Trial Configuration Metrics 87
C.3 Per-Trial Configuration Artifacts 87
C.4 Cognitive Walkthrough Step Counts by Persona 88
C.5 Variance Analysis and Sample Size 89

D.1 Information Available to Agent by Condition 96

Abstract

Saturn: Zero-Configuration AI Service Discovery

Joseph Perrello

Generative AI services gate access behind per-user subscriptions, API keys, and manual endpoint configuration—barriers that scale with the number of users and applications on a network. Saturn provisions AI the way printers and speakers already provision themselves: through Multicast DNS (mDNS) and DNS-based Service Discovery (DNS-SD), protocols that ship on every major operating system. An administrator registers AI endpoints under the service type `_saturn._tcp.local.`; every device on the network segment discovers them without accounts, credentials, or configuration files. Seven implementations across four languages and four independent mDNS libraries interoperate on this service type with no shared discovery code, demonstrating that the protocol specification—not a reference implementation—carries the design. A cognitive walkthrough across three user personas (administrator, developer, end user) measures a 53% reduction in total configuration steps compared to traditional per-user API key distribution, with developer effort dropping 79% and end-user payment friction eliminated entirely. Cloud-facing deployments rotate ephemeral API keys every five minutes; a captured credential expires before an attacker leaves the building. Saturn's broadcast model inherits known mDNS privacy risks—passive enumeration, device-name leakage—and enterprise WiFi access-point isolation blocks multicast entirely, confining viable deployment to home, office, and lab networks. These tradeoffs are documented in existing literature and addressable without destroying the zero-configuration property.

Acknowledgments

Adam Smith let me chase a strange idea—putting AI on mDNS—and then made me defend every piece of it. Ram Sundara Raman kept asking what happens when someone on the network lies, which is how Saturn got a real threat model instead of a hand-wave. Peter Biehl and the graduate division kept the paperwork from burying me.

Saturn reuses mDNS and DNS-SD almost unchanged. Stuart Cheshire and Marc Krochmal designed those protocols; the IETF DNSSD working group refined them over two decades. All Saturn contributes is one new service type. The foundation was already solid.

This project started because my classmates—people in a graduate CS program—could not use AI tools without API keys they did not have. Not a skill problem. A plumbing problem. I wanted to fix the plumbing.

Chapter 1

Introduction

1.1 The Problem: AI Access as Configuration Burden

Lewis Strauss predicted in 1954 that atomic energy would become "too cheap to meter" [39]. Generative AI landed with the same promise and the same fine print. ChatGPT Plus costs $20 per month [31]; Claude Pro matches it [1]. A researcher who subscribes to both pays $40 before usage-based tiers push the bill past $100. Price is the visible cost. Underneath sits a structural one: every AI application demands its own endpoint URL, its own API key, its own model selection—configured independently, stored independently, broken independently.

Syed et al. documented the architecture behind this friction: REST endpoints gated by per-user tokens, independently configured per application [40]. Open WebUI [30], LibreChat [22], and Jan [17] serve the same backend but share no configuration—three separate URL fields, three separate key stores, three separate failure points.

Those credentials leak and persist indefinitely because no one revokes them—a pattern Chapter 2 documents across both individual repositories [23] and organizational key inventories [35].

The per-user subscription model turns AI access into a monthly bill. The per-application configuration model turns it into ongoing labor. Both scale with the number of people involved. Saturn targets the structural version of this problem: the configuration burden that grows with every user and every application on a network.

1.2 The Vision: AI as Network Infrastructure

Apple shipped Bonjour in 2002. Within a year, printers appeared on Macs without drivers or IP addresses—the same Multicast DNS (mDNS) and DNS-based Service Discovery (DNS-SD) protocols now behind AirPrint and Chromecast (Chapter 2). RFC 1122 captured the aspiration: "It would be ideal if a host implementation of the Internet protocol suite could be entirely self-configuring" [5].

Language models missed this transition. Every chat application, every coding assistant, every transcription tool asks the same questions on first launch: which provider? Which key? Which endpoint? An address, a credential, a protocol dialect—the same three fields printers already broadcast over mDNS.

The precedent extends beyond desktops—eduroam provisions WiFi across hundreds of campuses without per-device configuration. Saturn applies the same principle to language models: join a network and the AI services on it appear. No account creation, no credit card, no API key management. The administrator who configures the services absorbs the complexity once; every other user inherits the result. The cost is scope: discovery reaches only the local network segment. Enterprise networks that enable AP isolation—common at universities, Saturn's strongest motivating environment—block the multicast traffic discovery depends on, a constraint Chapter 5 examines in detail.

1.3 Saturn in Brief

Under the service type `_saturn._tcp.local.`, servers register OpenAI-compatible endpoints; clients browse the service type and connect without setup. The protocol separates announcement from traffic: beacons broadcast metadata—where services live, how to authenticate—but never proxy API calls **D3**. Everything a client needs—endpoint URL, API type, credentials, routing priority—fits inside standard DNS-SD TXT records, and no side channel carries configuration.

The most constrained deployment runs on a $30 GL.iNet travel router alongside DHCP and DNS **A7**, demonstrating that AI provisioning fits in the network infrastructure layer—managed by the same init system, configured through the same web interface.

1.4 Thesis Statement

We can and should provision generative AI at the network level. Saturn demonstrates this through three claims. First, zero-configuration AI provisioning is feasible: mDNS/DNS-SD carries everything a client needs to discover and connect to AI services without manual setup. Second, network-level provisioning reduces total configuration effort: one administrator configures services once, eliminating per-user key distribution, per-application endpoint entry, and the entire billing integration stack. Third, the security trade-offs of broadcasting AI service metadata on a local network are documented in existing mDNS literature and addressable without destroying the zero-configuration property.

Saturn is a systems contribution. The burden it targets falls unevenly: socioeconomic status predicts who benefits from language technology [3], and per-user sub-

scriptions compound the disparity (Section 6.8). That inequity motivates the work and contextualizes its impact, but the thesis stands on technical merit: a working protocol, nine implementations, and measured reductions in configuration labor.

1.5 Contributions

Saturn's primary contribution is a protocol specification: a TXT record schema defining how AI services announce endpoint location, authentication credentials, API compatibility, and routing priority over mDNS/DNS-SD. Nine components across four languages test whether the specification—not the reference implementation—carries the design (Chapter 4). Seven resolve `_saturn._tcp.local.` through four independent mDNS libraries with no shared discovery code; two reuse the core package's mDNS stack to test additional integration patterns. A cognitive walkthrough across three user personas quantifies what network-level provisioning costs the administrator, the developer, and the end user (Chapter 5).

1.6 Organization

Chapter 2 surveys service discovery protocols and the AI applications Saturn targets. Chapter 3 specifies the Saturn protocol: service type, TXT record schema, beacon architecture, and ephemeral credential lifecycle. Chapter 4 walks through each implementation—server-side beacons, client libraries, application integrations, and the router deployment. Chapter 5 evaluates Saturn against the three claims through requirements analysis, a cognitive walkthrough, a heuristic evaluation, and an integration opportunity analysis of 42 existing applications. Chapter 6 discusses limitations, security tradeoffs, and directions for future work. Chapter 7 concludes.

Appendix B documents where Anthropic's Claude contributed to this work and where we overruled it.

Chapter 2

Background

The Introduction identified the configuration burden: every AI application demands its own endpoint, credentials, and model selection. This chapter explains why that burden persists at the protocol level, introduces the discovery mechanism Saturn adopts, and positions the security trade-offs that constrain the design.

2.1 The AI Service Landscape

Section 1.1 introduced the per-user access model behind AIaaS: REST endpoints gated by individual API keys and billing accounts [40]. This section examines the technical structure underneath that model—what the API looks like, how configuration fragments across applications, and why static credentials fail at scale.

2.1.1 The OpenAI-Compatible API

A de facto standard has emerged. OpenAI defined a REST API with three core endpoints: `POST /v1/chat/completions` for text generation, `GET /v1/models` for model enumeration, and `GET /health` for liveness checks. Ollama [27], LiteLLM [4], Open-

Router [32], and dozens of other providers implement this interface.

The API does not prescribe who runs the server—cloud provider, university data center, single-board computer running quantized models—only how clients talk to it. Applications that support "custom OpenAI endpoint" configuration already interoperate with any compliant backend. The wire format is settled; the unsolved problem is how a client finds a server in the first place.

2.1.2 Configuration Fragmentation

The API standard specifies how to talk to a provider but not how to find one. Each application assumes it manages its own provider relationship: its own endpoint URL, its own API key, its own model list. Chat interfaces, coding assistants, and voice transcription tools all implement the same OpenAI-compatible client independently. Adding a second AI application to a network doubles the configuration effort; adding a third triples it. Configuration scales linearly with application count, not with the number of services available.

Voice pipelines expose the problem in miniature. A user who pairs a local Whisper instance for transcription with a cloud language model for post-processing configures two models, two credentials, and two failure points—for what the user perceives as a single capability [36]. The models sit on the same network; neither knows the other exists.

2.1.3 Static API Key Fragility

Section 1.1 noted that credentials leak and persist. The scale is worse than the anecdote suggests. Meli et al. scanned GitHub and found secrets in over 100,000 repositories; 81% of those keys remained live after the owners were notified [23]. Their best auto-

mated detector caught only 25% of leaks, leading them to conclude that "all mitigations act too late"—the key is compromised before any scanner runs. Qazi documented the organizational side: most corporations cannot inventory which keys they have issued, let alone which remain active [35]. The failure compounds because static keys have no expiration by default; they persist until someone manually revokes them, and Meli's data shows that rarely happens even after notification.

2.2 mDNS and DNS-SD

mDNS resolves names without a server [10]. DNS-SD advertises services using standard DNS record types [9]. Together they let a device discover what services exist on its network segment using only multicast queries and standard DNS record types—no authority, no accounts, no configuration. Guttman defined zero-configuration networking as "direct communications between two or more computing devices via IP" without manual setup [16].

mDNS/DNS-SD has four defining properties, each carrying an inherent cost. It requires no infrastructure—no server, no authority, no accounts—and therefore no cross-subnet reach. It operates at link-local scope, confining queries and responses to a single network segment; enterprise access-point isolation can block multicast entirely (Section 2.5). It carries structured metadata in TXT records—arbitrary key-value pairs—but caps each string at 255 bytes. It ships preinstalled (Bonjour on macOS and Windows, Avahi on Linux), which means multicast traffic is visible to every device on the link, including passive observers.

2.2.1 Multicast DNS

RFC 6762 specifies Multicast DNS [10]. Instead of querying a designated resolver, the client multicasts a DNS query to 224.0.0.251 on port 5353 (ff02::fb on IPv6). Every device on the link hears the query. Whichever device owns the requested name responds with its IP address. Names end in `.local`. The wire format uses ordinary DNS record types: A, AAAA, SRV, TXT, PTR. Two properties distinguish mDNS from conventional DNS: transport (multicast instead of unicast) and scope (link-local instead of global). Siddiqui et al. placed mDNS alongside link-local addressing [8] and DNS-SD as the three pillars of zero-configuration networking [37].

2.2.2 DNS-Based Service Discovery

mDNS resolves names. DNS-based Service Discovery (DNS-SD), specified in RFC 6763, answers the next question: what services of a given type exist on this network [9]?

Three record types chain together. A **PTR record** maps a service type—`_saturn._tcp.local.`—to instance names. This is the enumeration step: the client learns which instances exist. An **SRV record** provides hostname and port for each instance. A **TXT record** carries key-value metadata: deployment type, API compatibility, credentials, priority. A client queries PTR, receives instance names, then resolves SRV and TXT for each instance.

DNS-SD rides on mDNS for link-local scope or on unicast DNS for wide-area deployments. Saturn uses mDNS only, confining discovery to the local network segment.

2.2.3 TXT Records and Metadata

TXT records carry key-value pairs encoded as strings. The DNS wire format caps each string at 255 bytes; a record can hold multiple strings, each independently limited.

Saturn uses TXT fields to carry protocol version, deployment type (local or cloud), API type (openai-compatible), base URL, priority, ephemeral key, key rotation interval, and feature flags. A client that resolves these fields knows enough to connect—no follow-up query, no out-of-band configuration needed. The 255-byte ceiling constrains field sizes: an ephemeral key gets roughly 240 usable characters after accounting for the key name and delimiter. Saturn's metadata fits within these limits.

2.2.4 Platform Adoption

On macOS, Bonjour powers AirPrint, AirPlay, and AirDrop [2]. Linux distributions run Avahi, an open-source mDNS/DNS-SD implementation [34]. Microsoft added a native mDNS responder in Windows 10.

Siljanovski et al. extended mDNS into IoT sensor networks, arguing that "when possible it would be better to adopt preexisting Internet protocols" rather than design new discovery mechanisms [38]. VS Code uses mDNS to locate remote debugging targets without manual IP address entry [24]. Kim and Reeves traced the DNS deployment footprint across consumer devices, documenting mDNS adoption beyond its desktop origins [20].

Saturn's most direct precedent is a community pull request proposing mDNS advertisement for Ollama, a local language model server [28]. It was never merged. `_saturn._tcp` instead of `_ipp._tcp`, same machinery underneath.

2.3 Alternative Discovery Protocols

Four other protocols have competed for service discovery on local networks. The problem domain from Section 2.1 constrains the choice: the mechanism must carry structured metadata (model names, credentials, API types), must not require infrastructure

that home and lab networks lack, and must ship preinstalled so that consumer devices can discover services without additional software.

NetBIOS resolves host names through broadcast queries on a flat namespace [15]; it carries no service metadata and never shipped outside Windows. Universal Plug and Play (UPnP) [29] carries structured metadata over SSDP and supports cross-subnet discovery—capabilities mDNS lacks—but two decades of remote code execution vulnerabilities have led most networks to disable it, and no preinstalled stack remains reliably available. DLNA defined interoperability profiles for streaming media using UPnP discovery [12]; the consortium dissolved in 2017, and the specification never extended beyond audio and video. WS-Discovery [26] carries structured metadata and supports managed namespaces that would simplify multi-tenant deployments, but it targets enterprise environments with infrastructure that home and lab networks lack.

mDNS/DNS-SD is the only production-deployed protocol that satisfies all three constraints. The cost is real: Saturn inherits mDNS's link-local scope and cannot reach services across subnets—a limitation UPnP and WS-Discovery do not share.

2.4 The DHCP Parallel

DHCP exemplifies network-level provisioning [13]. A device broadcasts DISCOVER; a server replies with an IP address, gateway, DNS server, and lease duration; the client is online without manual configuration. One administrator configures the DHCP server; every device on the network inherits the result.

DHCP provisions a single resource (an IP address) in a single exchange at connection time. Services that appear and disappear after the initial handshake—or that require structured metadata beyond an address—fall outside its model. AI services exhibit both properties: they vary in number, rotate credentials, and carry metadata (model names,

API types, authentication tokens) that DHCP's fixed-field format cannot express.

2.5 Security Context

Saturn broadcasts service metadata—including, for cloud deployments, API credentials—to every device on the local network. This design choice positions Saturn opposite the prevailing security philosophy of the past decade.

2.5.1 Zero-Trust and BeyondCorp

Ward and Beyer described Google's BeyondCorp model: no trusted internal network, every request authenticated regardless of origin, device inventory and access control enforced at every boundary [41]. Osborn et al. extended it with tiered trust and granular access policies [33].

Saturn deliberately diverges. Its target environments—home networks, lab networks, small office deployments—lack the infrastructure BeyondCorp requires: managed device fleets, identity providers, access control engines. Campus WiFi users do not carry managed devices. Coffee shop patrons do not have identity providers. Saturn trades enterprise-grade verification for zero-configuration access. This is a deliberate design choice for the target environment, not an oversight. Chapter 5 evaluates the consequences.

2.5.2 mDNS Privacy and Vulnerability Surface

Kaiser and Waldvogel demonstrated that a passive observer can enumerate mDNS services, track devices, and correlate metadata with users [18]. Könings et al. found that 59% of mDNS device names in the wild contain real user names—announcements

functioning as personally identifiable information [21].

Saturn adds AI-specific metadata to this existing exposure surface. Service names, model lists, deployment types, and—for cloud beacons—ephemeral credentials all travel in cleartext multicast. The threat model is well-documented because Saturn inherits it from two decades of mDNS deployment. Kaiser and Waldvogel proposed privacy-preserving DNS-SD modifications, including hash-based instance names, that maintain backward compatibility [19]; these represent viable future extensions. Chapter 5 evaluates Saturn against these known weaknesses.

2.5.3 The Gap Saturn Addresses

The access models described in Sections 2.1 and 2.1.3—cloud AIaaS and local inference alike—require per-application endpoint configuration. Neither treats AI services as discoverable network resources.

mDNS/DNS-SD has been available for over two decades, yet no system applies it to AI services. The gap persists for two reasons. First, AI services emerged in a cloud-first context where discovery was unnecessary: a single provider URL, hard-coded or environment-configured, sufficed. Second, AI endpoints carry authentication credentials—a requirement printers and media servers never had—and broadcasting credentials over multicast introduces a security surface that existing mDNS applications avoid. The result is a class of network services that remains manually configured despite a mature, preinstalled discovery substrate. Chapter 3 specifies a protocol that addresses this gap.

Chapter 3

Design

Printers announce themselves over mDNS; AI services do not. Saturn closes that gap with a protocol specification—a service type, a TXT record schema, and a discovery sequence that any mDNS-capable device can follow **V1**.

This chapter specifies Saturn's protocol in sufficient detail that an independent implementor can build a conforming client or server from scratch. Section 3.1 states the properties the system must achieve. Section 3.2 names the roles that interact with Saturn and defines their responsibilities. Section 3.4 introduces the four core abstractions. Section 3.5 gives the wire-level protocol specification. Section 3.6 explains the architectural choices and the alternatives they beat.

3.1 Goals

Saturn serves one purpose: make AI endpoints available to every device on a local network without per-device configuration. Zero-configuration (**D2**) means operational without end-user setup of URLs, API keys, or account credentials.

Zero-configuration for consumers. Any application on the network segment dis-

covers available AI services through standard multicast DNS [10] **A3**.

Minimal configuration for administrators. One person—the administrator—bears the entire configuration burden: obtaining API credentials, deploying beacons, naming services, and setting routing priorities. This asymmetry is deliberate. Saturn concentrates complexity on one person so every other person on the network does zero work **A4**.

Vendor independence. The protocol binds to an interface standard—the OpenAI-compatible REST API—not a company. Anthropic could raise prices, OpenRouter could shut down, a university might switch providers mid-semester. Any backend behind the OpenAI-compatible API qualifies as a Saturn endpoint. The specification defines a service type, a TXT record schema, and endpoint contracts; it does not prescribe a language, an mDNS library, or an SDK.

Bounded credential exposure. Broadcasting API keys to every device on a network segment hands credentials to anyone listening. Kaiser and Waldvogel documented this threat for mDNS-SD: "every machine in the same network will automatically receive all the announcement traffic" [18]. Saturn limits the damage through time: cloud-facing beacons rotate keys every five minutes, and a stolen credential expires before an attacker reaches a different network **A5**.

3.2 Audiences

Configuration labor does not disappear. Saturn distributes it unevenly across three roles.

Administrators carry the full weight. They obtain API keys from providers, deploy beacons on the network, write TOML configuration files or fill in LuCI web forms, name services, and assign routing priorities. The zero-configuration experience that consumers enjoy exists because this person configured everything in advance. A uni-

versity sysadmin, a roommate with a GPU, a teacher sharing a cloud key for a class exercise—all fill this role **A4**.

Application developers write four lines of code: install, import, `discover()`, connect. No API key management, no billing integration, no provider selection logic. Open-source developers benefit most: they ship Saturn-aware applications without bundling API keys or forcing users to manage provider accounts. Saturn removes the key from the repository entirely, sidestepping the leakage documented in Section 2.1.3 **A12**.

End users open an application, find AI available, and use it. They never see Saturn's name. The closest analogy is AirPlay: join a network, open a compatible app, the service is already there **A13**.

AI **providers**—OpenAI, Anthropic, OpenRouter, DeepInfra—change nothing. Saturn consumes their existing APIs. A provider that serves the OpenAI-compatible REST API participates in Saturn without writing a single line of Saturn-specific code.

3.3 Design Origins

Three fictional personas—a sysadmin (Derek), an end user (Mira), and a malicious developer (Jordan)—traced the provisioning lifecycle before implementation began. Each later mapped to a cognitive walkthrough persona in Chapter 5.

Derek and Mira established the asymmetric complexity trade-off **V2**: Derek configures one TOML file, one environment variable, one command; Mira connects to WiFi and finds AI available without accounts, keys, or payment. If Mira has to configure something, the zero-configuration claim fails. Mira's scenario also crystallized the trust question Chapter 6 examines—she does not know that her photo left her device and traveled to a cloud endpoint.

Jordan's scenario drove the ephemeral key architecture. Before Jordan, Saturn servers held permanent API keys in configuration files. After Jordan, beacons mint credentials with 10-minute lifetimes and 5-minute rotation intervals. A copied key expires before an attacker can exploit it at scale. Jordan's scenario shaped Claim 3 **V3**: security trade-offs are known and addressable.

The personas did not surface every constraint. AP isolation—enterprise WiFi blocking multicast between clients—appeared only during campus deployment testing (Section 5.5.4). TXT record schema drift emerged during the heuristic evaluation (Section 5.4). Three personas covered provisioning; they missed the network infrastructure constraints that shape where Saturn can operate.

3.4 Concepts

Four abstractions carry the protocol: endpoints, beacons, priorities, and ephemeral keys.

3.4.1 Endpoints

A Saturn endpoint (**D1**) is any HTTP service that implements the OpenAI-compatible REST API. Ollama, OpenRouter, LiteLLM, vLLM, and dozens of other providers already serve this interface—JSON messages in, completions out. Saturn adopts it wholesale. The endpoint may run on a cloud provider, a university data center, or a single-board computer running quantized models. If it responds to the three paths defined in Section 3.5.3, it qualifies. No code changes, no Saturn-specific integration.

The protocol does not prescribe who runs the server. A university IT department, a hobbyist on a Raspberry Pi, or a commercial provider all produce valid endpoints. Saturn binds to the interface, not the operator.

3.4.2 Beacons

A beacon (**D3**) registers a service under `_saturn._tcp.local.`, publishes metadata in DNS TXT records, and answers discovery queries. It does not touch API traffic. Clients discover endpoints through the beacon and connect to the advertised URL directly—the beacon announces where to go and how to authenticate but never sees a prompt or a response.

That separation has consequences. Kill the beacon and active connections survive; clients already holding an endpoint address keep using it. The beacon's only job is DNS registration and credential management, which is why it runs on hardware as constrained as a travel router (Chapter 4, Section 4.2.1).

Two types exist:

- A *cloud beacon* announces a remote provider (OpenRouter, DeepInfra) and distributes ephemeral API keys (Section 3.4.4). The beacon authenticates with the provider using the administrator's permanent credential, mints short-lived sub-keys, and broadcasts them in TXT records. The permanent credential never touches the wire.

- A *local beacon* announces a service running on the LAN—an Ollama instance, a vLLM server—and monitors its health. When the service goes down, the beacon pulls its DNS entry so clients stop discovering a dead endpoint.

3.4.3 Priorities

A network might run a local GPU and keep a cloud API as fallback. Priority—a number in each service's TXT record—determines which wins. Lower values beat higher: priority 1 over priority 50. The semantics follow DNS SRV record convention, so network

engineers recognize the pattern immediately.

The administrator who wants local-first inference assigns priority 1 to the GPU and priority 50 to the cloud backend. Users get whichever is available without choosing or knowing the difference. Priority is policy, not performance—the administrator decides what "best" means for this network. If the priority-1 service goes down, clients automatically fall through to priority 50. No client-side configuration changes; failover emerges from the priority ordering.

3.4.4 Ephemeral Keys

Broadcasting a permanent API key to every device on a network segment invites abuse. Someone records a key from a beacon, walks out of the building, and uses it from a coffee shop. Saturn limits the damage through time **A5**.

Cloud beacons generate keys that live ten minutes on the provider's server and rotate every five. The lifecycle works as follows:

1. The beacon authenticates with the provider's key-provisioning API using the administrator's permanent credential.

2. The provider returns a scoped, time-limited sub-key (an API key for OpenRouter, a scoped JWT for DeepInfra).

3. The beacon broadcasts the sub-key in the mDNS TXT record.

4. Five minutes later, the beacon mints a new key, broadcasts it, and revokes the old one through the provider's API.

5. Old and new keys overlap briefly so clients never lose connectivity mid-request.

6. On shutdown, the beacon deletes the current key.

Record a credential from a broadcast, leave the network, try the key ten minutes later—revoked. The mechanism resembles Kerberos ticket distribution: time-limited credentials scoped to a session, applied at the application layer through provider key-provisioning APIs.

Automatic expiry eliminates the dominant failure mode documented in Section 2.1.3: leaked credentials that persist because no one revokes them.

Local deployments skip the mechanism entirely. An Ollama instance on a private network needs no authentication; the TXT record's `features` field says so. Cloud deployments require that the provider support programmatic key creation and deletion—OpenRouter and DeepInfra do; providers without provisioning APIs cannot participate in ephemeral key rotation.

3.5 Protocol Specification

Combined with RFC 6762 (mDNS) [10] and RFC 6763 (DNS-SD) [9], what follows is sufficient to build a conforming implementation from scratch.

3.5.1 Service Type

Saturn services register under the DNS-SD service type `_saturn._tcp.local.`. The underscore-prefixed format follows RFC 6763 naming conventions for DNS-SD service types [9]. The `_tcp` designator indicates that clients connect via TCP after discovery. The `.local.` suffix confines discovery to the link-local multicast domain defined by mDNS [10].

3.5.2 Record Structure

Saturn reuses the DNS-SD record chain described in Chapter 2. Three DNS record types cooperate per service instance:

- A **PTR record** maps `_saturn._tcp.local.` to instance names—the enumeration step. A single PTR query returns every Saturn service on the network segment.
- An **SRV record** per instance provides hostname and port—the location step.
- A **TXT record** per instance carries the metadata: deployment type, API compatibility, priority, and—for cloud beacons—an ephemeral credential.

Once a client has all three records for a service instance, it can connect without further queries **A8**.

Table 3.1 specifies the TXT record fields that constitute the Saturn protocol.

Optional extensions exist for model enumeration (`models`), capability advertisement (`capabilities`), context window size (`context`), and pricing tier (`cost`). Clients ignore fields they do not recognize, following the extensibility principle of DNS-SD [9]. Every value is a UTF-8 string; every key-value pair must fit the 255-byte DNS TXT string limit defined by RFC 6763.

Six required fields plus two conditional fields carry all the information a client needs to discover, authenticate with, and connect to any Saturn service **A8**. Three distinct API backends—Ollama (no authentication), DeepInfra (scoped JWT), and OpenRouter (ephemeral API key)—operate through this same schema **A9**.

3.5.3 Endpoint Expectations

Three HTTP paths are mandatory for any Saturn endpoint (Table 3.2).

Table 3.1: Saturn TXT Record Schema

Field	Presence	Description
`version`	Required	Protocol version (currently `1.0`).
`deployment`	Required	`cloud` for remote providers; `local` for on-network inference.
`api_type`	Required	API compatibility identifier (e.g., `openai`, `ollama`).
`api_base`	Required	Base URL for API requests. For local services: `http://192.168.1.5:11434/v1`. For cloud beacons: `https://openrouter.ai/api/v1`.
`priority`	Required	Numeric routing preference; lower values indicate higher priority.
`features`	Required	Feature flags: `ephemeral_auth` for cloud beacons with key rotation; `network_proxy` for local services.
`ephemeral_key`	Conditional	Temporary API credential; present when `features` includes `ephemeral_auth`. Must fit within 240 usable bytes of the 255-byte TXT string limit.
`rotation_interval`	Conditional	Key rotation interval in seconds; present alongside `ephemeral_key`.

These three paths match the subset of the OpenAI REST API that every major provider and local inference engine already implements. Ollama serves them natively. OpenRouter, DeepInfra, LiteLLM, and vLLM serve them through their OpenAI-compatibility layers. Saturn requires no custom endpoint—only the endpoints that already exist.

Cloud beacons present a subtlety: the `api_base` in their TXT record points to the upstream provider, not to the beacon itself. Clients extract the `ephemeral_key` from the TXT record and authenticate with the provider directly.

3.5.4 Discovery Process

Discovery completes in four steps:

Step 1: Browse. The client multicasts a DNS-SD PTR query for `_saturn._tcp.local.`,

Table 3.2: Required Saturn Endpoint Paths

Method	Path	Behavior
`GET`	`/v1/health`	Returns a JSON object with a `status` field. Clients issue this request to verify service availability before and during use.
`GET`	`/v1/models`	Returns a JSON array of available model identifiers in the OpenAI list-models format.
`POST`	`/v1/chat/completions`	Accepts a JSON body containing a `messages` array and `model` identifier. Returns a chat completion response. Must support both streaming (server-sent events) and non-streaming modes.

requesting the names of all Saturn service instances on the local network segment.

Step 2: Resolve. For each discovered instance, the client queries the corresponding SRV and TXT records. The SRV record yields a hostname and port; the TXT record provides deployment type, API base URL, priority, and—for cloud beacons—an ephemeral credential.

Step 3: Sort. The client orders discovered services by the `priority` field in ascending order (lower values first). Ties may be broken by implementation-defined criteria such as the `cost` field or random selection.

Step 4: Select. The client issues a `GET /v1/health` request to the highest-priority service. If the service responds, the client connects. If not, the client proceeds to the next service in priority order. Discovery fails only when no service responds.

If a connected service drops—failed health check, connection timeout, or HTTP error—the client restarts from Step 1. This four-step process requires no configuration from the consumer **A3**.

3.6 Architecture Decisions

3.6.1 Beacon, Not Proxy

A proxy would simplify client integration—one URL, one connection—but the proxy sees every prompt and response, creates a single point of failure for the entire network, and must carry throughput it did not generate. Beacons sidestep all three problems: clients discover endpoints at startup and connect directly, so the beacon never processes API traffic.

The trade-off is real. Applications that lack mDNS capability cannot consume beacons directly. Saturn addresses this through bridge patterns: a small local process handles discovery and presents a stable localhost endpoint. The VLC extension (Chapter 4) demonstrates this pattern. The bridge runs on the user's machine as the user's own process, keeping the beacon's architecture clean.

3.6.2 Multicast, Not Registry

A centralized registry would cross subnets and resolve queries faster. It would also need an operator, a domain name, TLS certificates, and an availability plan. mDNS requires the network and nothing else—the zero-configuration criterion established in Section 2.2.

The scope limitation is deliberate. mDNS confines discovery to a single broadcast segment. Saturn targets home networks, campus WiFi, small offices—places where one segment *is* the whole network **V1**. Multi-site deployments or cross-subnet discovery would require a registry or DNS-SD wide-area extensions—infrastructure Saturn intentionally avoids.

This constraint has a hard edge: enterprise WiFi networks that enable AP isola-

tion block multicast between clients. Saturn cannot function on such networks. Section 5.5.4 confirms the failure on two institutional deployments **A17**. Saturn works on its target networks (home, office, lab) but fails on institutional networks with client isolation enabled.

3.6.3 OpenAI-Compatible API, Not a Custom Protocol

Saturn could define its own RPC format, optimized for AI workloads. Instead it adopts the OpenAI-compatible REST API because that interface already dominates the ecosystem. Every major AI application—Cursor, Continue, Open WebUI, Jan, Aider—already speaks this protocol. Building on it means Saturn requires zero client-side code changes for any application that supports a custom OpenAI endpoint.

The cost: Saturn handles only text completion, the capability the OpenAI API standardizes most widely. Image generation, audio synthesis, and embedding computation use different API shapes. Supporting other modalities would require new TXT schema extensions and endpoint contracts—but not a redesign of the mDNS/DNS-SD discovery layer.

3.6.4 Protocol, Not Implementation

The specification defines a service type, a TXT record schema, and endpoint contracts. It does not prescribe a language, an mDNS library, or an SDK. Any language with an mDNS stack can implement a conforming client or server—interoperability follows from the wire format, not from shared code **A10 A2**.

Chapter 4

Implementation

The design chapter specified what Saturn must do. This chapter shows the running system. Nine components across four languages—Python, Rust, TypeScript, and Lua—implement the protocol from Chapter 3. Each targets a different integration pattern, audience, or hardware constraint. The chapter presents the core package first, then the router deployment, then seven client integrations organized by the pattern each uses to consume mDNS-discovered services.

Table 4.1 summarizes the implementation landscape; the Pattern column references the six integration strategies defined in Section 4.3.

Table 4.1: Saturn Implementation Components

Component	**Language**	**mDNS Library**	**Pattern**
Core Package	Python	`zeroconf`	Server + client library
Saturn Router	Rust	`mdns-sd`	Embedded beacon
AI SDK Provider	TypeScript	`multicast-dns`	Native SDK
Open WebUI Plugin	Python	`Zeroconf`	Application plugin
MCP Server	Python	`zeroconf` (via core)	Tool protocol bridge
VLC Extension	Lua/Python	`dns-sd` CLI	Subprocess bridge
OpenCode Fork	TypeScript	`multicast-dns` (via SDK)	Application fork
Jan Proxy	Python	`zeroconf` (via core)	Localhost proxy
Aider Wrapper	Python	`zeroconf` (via core)	Env-var injection

Saturn's original contribution is the beacon logic, discovery client, and per-application integration code; the mDNS libraries, HTTP frameworks, and packaging tools in Table 4.1 are reused infrastructure.

4.1 Core Python Package

The `saturn-ai` package on PyPI (v1.0.1) implements both sides of the protocol: server-side beacon advertisement and client-side service discovery. Python is the implementation language because the AI tooling ecosystem it targets—Ollama's API bindings, cloud provider SDKs, FastAPI for proxy servers—centers on it. The `zeroconf` library handles mDNS operations; it provides async browsing, cross-platform registration, and no native dependencies.

4.1.1 Discovery Layer

The `SaturnAdvertiser` class wraps `zeroconf`'s `ServiceInfo` and `register_service()` calls to handle beacon registration. Before registering, the advertiser runs a two-second discovery scan to detect priority values already in use on the network and increments until it finds an unused slot—collision avoidance that the protocol specification leaves to implementors.

On the client side, `discover()` wraps `zeroconf`'s async service browser to execute the resolution sequence from Section 3.5.4. A one-second settle-time—the browser waits for a quiet window with no new responses before returning—trades latency for completeness. On a network with no Saturn services, the caller blocks for the full timeout before receiving an empty result.

4.1.2 Server Types

Three server types exist because backend providers differ in authentication: local Ollama requires none, cloud providers like OpenRouter issue scoped sub-keys, and test environments need canned responses. Each type exercises a different part of the TXT record schema.

The **Ollama server** translates between Ollama's native API and the OpenAI-compatible format. The beacon confirms the Ollama instance is alive via `/v1/health`, pulls loaded models from `/api/tags`, and packs both the endpoint and the model list into a single TXT record. Clients learn what is available at browse time without a follow-up query.

The **OpenRouter server** exercises the ephemeral credential lifecycle from Section 3.4.4. OpenRouter's provisioning API supports scoped sub-key creation with server-side expiry and programmatic deletion—the two operations the lifecycle requires. The beacon calls OpenRouter's key management endpoint to mint and revoke; clients see only the current key in the TXT record.

The **fallback server** returns canned responses from hardcoded models. It exists for testing: a developer validates Saturn discovery end-to-end without running Ollama or configuring cloud credentials.

4.1.3 Configuration System

Administrator configuration uses TOML files. Each file defines a service with fields for name, deployment type, API compatibility, priority, upstream URL, and beacon settings. Built-in configurations ship with the package for common providers (Ollama, OpenRouter, DeepInfra). User configurations in `~/.saturn/services/` shadow built-in defaults.

The CLI entry point `saturn run <name>` loads a configuration, validates it, starts

a FastAPI proxy server, registers the service on mDNS, and runs until interrupted. One command provisions one service. The entire administrator workflow—from configuration to running beacon—requires a TOML file and an environment variable for the API key **A4**.

4.2 Embedded Deployment

4.2.1 GL.iNet Router Deployment

The GL.iNet deployment places Saturn on the same device that runs DHCP and DNS—the infrastructure-layer goal identified in Section 3.1.

OpenWRT Integration

An administrator manages Saturn through the same interfaces used for the router's other network services. A procd init script integrates Saturn into OpenWRT's service manager—the same framework that manages dnsmasq, hostapd, and the firewall. Configuration goes through UCI (Unified Configuration Interface), the standard tool an administrator uses for DHCP pools and firewall rules. Each `config service` UCI section spawns a separate procd-managed process with per-service JSON configuration files protected by `chmod 600` permissions. The init script auto-downloads the correct binary from GitHub releases if it is missing, so the administrator configures UCI and starts the service without manual file transfer.

A LuCI web interface provides form-based service management. Dynamic field visibility hides cloud-specific options (API key, rotation interval) when the deployment type is "network" and vice versa. Input validation enforces name format, URL structure, and the constraint that key expiration must exceed rotation interval. Status badges

poll every ten seconds, showing live health for each configured service.

Rust Binary

The router's hardware constraints—a single-core MIPS32 processor with 16 MB of flash—rule out interpreted languages and large runtimes. The `saturn-router` binary uses Rust for zero-overhead static compilation and the `mdns-sd` crate for mDNS registration—a third library in a third language, exercising the vendor-independence goal from Section 3.1.

The main event loop ticks every 100 milliseconds. In ephemeral key mode, the loop rotates credentials on schedule, re-registers mDNS with updated TXT records, and deletes the previous key through the provider's API. In health-check mode, the loop monitors the backend service: when health fails, the beacon unregisters from mDNS so clients stop discovering a dead endpoint; when health recovers, the beacon re-registers. The `Drop` implementation on the provider struct ensures that ephemeral keys are deleted even on unexpected termination—defense-in-depth against leaked credentials.

Hardware Constraints

The target device—a GL.iNet GL-MT300N-V2—runs a single-core MIPS32 processor with 128 MB of RAM and 16 MB of flash. The binary cross-compiles to `mipsel-unknown-linux-musl` via Docker, targeting the MIPS32 soft-float architecture. The release profile minimizes binary size (`opt-level = "z"`, LTO, symbol stripping, `panic = "abort"`), and dependencies stay minimal—`attohttpc` instead of `reqwest`, hand-rolled argument parsing instead of `clap`—to fit within 16 MB of flash. The hardware is too constrained for inference but sufficient for the beacon workload described in Section 3.4.2.

Limitations

The release binary is approximately 700 KB. Flash is too small to store it, so the init script downloads the binary into RAM on each boot—a firmware upgrade or power loss triggers a fresh download, and the router must reach GitHub to restore service. At runtime, the binary's memory footprint is small relative to the 128 MB available, but the router runs dnsmasq, hostapd, and the firewall concurrently; Saturn competes for the same constrained pool. Multi-site deployments break this model: each location requires its own router with an independent TOML configuration and no synchronization mechanism. If the router fails, discovery stops for every client on that network until reboot completes and the binary re-downloads.

4.3 Client Integration Patterns

Each target application exposes a different extensibility surface—plugin architectures, tool protocols, configuration points, or nothing at all. Surveying the nine applications revealed six integration patterns, arranged below from deepest integration to broadest applicability. Deeper patterns yield better runtime behavior (automatic credential rotation, failover) but impose stricter requirements on the target:

1. **Native SDK**: The application imports a Saturn library that implements the host platform's provider interface. Automatic key refresh, failover, and circuit breaking—but the application must adopt a Saturn dependency.

2. **Application plugin**: A single file uploads into the application's native extension system. Discovery runs inside the application's process—but only applications with plugin architectures qualify.

3. **Tool protocol bridge**: An external process connects via a standard tool protocol (MCP/stdio). One config entry replaces per-provider key management—but the application must support the bridging protocol.

4. **Subprocess bridge**: A local process performs discovery and exposes a stable localhost endpoint. Works with any application that supports custom endpoints—but adds a background process and a security surface.

5. **Localhost proxy**: A background process discovers Saturn services and exposes them as a local OpenAI-compatible endpoint. Works with any application that supports a custom base URL—but the application sees a cached snapshot of the network between proxy refreshes.

6. **Environment variable injection**: A wrapper discovers services, sets environment variables, and launches the target application unmodified. No code changes to the target—but the application sees a static snapshot of the network at launch time.

Each pattern trades integration depth for applicability breadth.

4.3.1 Native SDK Integration

Of the seven client applications in Table 4.1, only two qualify for this pattern: both are TypeScript applications built on Vercel's AI SDK.

The `ai-sdk-provider-saturn` npm package (v0.1.4) implements Vercel's AI SDK `ProviderV3` interface. It resolves endpoint configuration, authentication, and model routing entirely at runtime from mDNS broadcasts. A developer writes:

```
import { saturn } from 'ai-sdk-provider-saturn';
```

```
const result = await generateText({
  model: saturn('gpt-4o'),
  prompt: '...'
});
```

The `multicast-dns` npm library, a pure JavaScript implementation with no native dependencies, starts discovery on import. By the first `generateText` call, the provider holds a live, priority-sorted service registry populated from mDNS broadcasts.

The discovery layer follows the resolution sequence defined in Section 3.5.4, polling every five seconds and maintaining a live service registry. Stale services (not seen for twenty seconds) are removed automatically. Goodbye packets (TTL=0) trigger immediate removal. These intervals have costs: a newly registered service takes up to five seconds to appear in the registry, and a crashed service lingers for up to twenty seconds before clients stop routing to it.

When a request receives a 401 response—indicating an expired ephemeral key—the model layer sends a targeted TXT query, waits up to two seconds for the beacon to broadcast a fresh key, and retries with the new credential. The two-second blocking wait is a trade-off: it recovers transparently from key rotation, but concurrent requests during a rotation event each block independently.

A circuit breaker trips after three consecutive failures and enters a thirty-second cooldown before sending a half-open probe. When a circuit opens, the provider falls back to the next service in the priority list. If no healthy service remains, the request fails.

4.3.2 Application Plugin

The Open WebUI Saturn plugin (`owui_saturn.py`) tests whether a single file—no build step, no dependency declaration—can give an application full Saturn discovery. Open WebUI's Pipe plugin architecture accepts Python file uploads through its admin panel. The plugin browses for `_saturn._tcp.local.` services via Python's `Zeroconf` library, fetches models from each discovered endpoint, and presents them in the application's model picker prefixed with `SATURN/`.

The cache refreshes on a configurable TTL; services that appear between refreshes become available on the next cycle. Priority-based failover routes requests to alternative services when the primary fails.

Five settings—name prefix, discovery timeout, failover toggle, cache TTL, and request timeout—surface through Open WebUI's Valves architecture, the plugin system's only mechanism for runtime configuration. The administrator adjusts them from the web UI without editing code or files.

4.3.3 Tool Protocol Bridge

The `saturn-mcp` package exposes Saturn's discovery as MCP (Model Context Protocol) tools for AI coding assistants. Built on Anthropic's FastMCP framework and served over stdio transport, the server provides six tools: discover services, list models, find a service for a specific model, find a service with specific capabilities, send a chat completion, and get service details.

Every tool begins with the same call: `discover()` from the core Saturn package, wrapped in `asyncio.to_thread()` to bridge synchronous mDNS discovery into MCP's async handler context. The bridge adds per-call latency: each discovery blocks a thread pool slot for the settle-time duration (one second by default). For beacon ser-

vices, both `list_available_models` and `chat_completion` add `Authorization: Bearer` headers with the ephemeral key from the TXT record. The stdio transport requires one subprocess per assistant session, and the six tools expose Saturn's full service inventory to the assistant—any MCP-connected model can enumerate, query, and send completion requests to every discovered endpoint.

One JSON entry in the MCP client configuration replaces per-provider API key management across Claude Code, Cursor, Windsurf, and any MCP-compatible assistant **A3**.

4.3.4 Subprocess Bridge

VLC media player predates Saturn and has no mDNS capability, making it the test case for this pattern. Its users watch video—they do not debug APIs.

Architecture

VLC's extension system supports only Lua scripting with restricted HTTP capabilities (GET requests only, no POST), and Lua has no native mDNS support. Two layers work around these constraints.

Layer 1: A Python/FastAPI bridge service runs as a background subprocess launched by the Lua extension. The bridge performs mDNS discovery using macOS's `dns-sd` command-line tool via subprocess calls—browsing `_saturn._tcp.local.`, resolving service addresses, maintaining health checks, and exposing an OpenAI-compatible REST API on localhost (127.0.0.1). The bridge writes its port to a file that the extension reads, establishing inter-process communication without hardcoded ports.

Layer 2: The Lua extension creates a VLC dialog window, manages user interaction, formats requests as URL-encoded JSON payloads (working around Lua's GET-

only limitation), and sends them to the localhost bridge via HTTP. The extension extracts media metadata—title, artist, album, genre, playback timestamp—from VLC's input item API and includes it as context in every chat request.

The bridge is packaged into a standalone executable using PyInstaller, bundling the Python runtime so end users need not install Python separately. Cross-platform backgrounding handles Windows (`start /B`), macOS, and Linux process launching. Installation is copying one directory into VLC's extensions folder; activation is a menu click: View → Extensions → Saturn Chat.

The bridge introduces a security surface. The `/shutdown` endpoint has no authentication—any process on the local machine can terminate the bridge. The bridge binds to `127.0.0.1`, limiting network exposure, but does not validate the identity of discovered services. A rogue service broadcasting `_saturn._tcp.local.` on the same LAN would appear alongside legitimate ones. These are deliberate trade-offs for a consumer desktop application: the bridge runs as the user's own process on their own machine, and the threat model assumes a trusted local network.

4.3.5 Application Fork

When an application lacks both a plugin system and a compatible tool protocol, source modification is the last resort. Forking trades broad applicability for deep integration: the result behaves identically to a native SDK integration, but the changes are tied to a specific application version.

OpenCode is an open-source AI coding agent with a TUI, a web UI, an event bus, LSP integration, and provider-agnostic model routing. The Saturn fork integrates `ai-sdk-provider-saturn` as a bundled provider. When launched, the agent discovers Saturn services on the local network via mDNS and makes their models available

for code generation, editing, and agentic tool use.

The integration required changes across twelve files in four packages. A custom loader in the provider system creates a `createSaturn()` SDK instance, waits three seconds for mDNS responses, then fetches `/v1/models` from all discovered services. Each discovered service registers as a dynamic provider with its own model list. Background callbacks handle network changes: when a new service appears, the agent fetches its models and registers a new provider; when a service disappears, the agent removes it. Both events propagate through the application's event bus so the TUI and web UI update their model selectors in real time.

Integration surfaced six bugs, each rooted in the host application's assumption that providers are static. Discovery callbacks that add or remove providers at runtime broke provider deduplication, model list caching, and Bun's default fetch timeout for long-running inference streams.

4.3.6 Localhost Proxy

Jan tests the boundary case for Saturn adoption: if the protocol works with an application whose only extensibility is a URL field, then the proxy pattern is a universal fallback for any OpenAI-compatible client. Jan is a privacy-focused desktop chat application with no plugin system, no tool protocol support, and no source code access. The integration uses `local_proxy_client.py` from the core Saturn package, which discovers Saturn services via mDNS and exposes them at `http://127.0.0.1:8080/v1`. Jan connects to this address as if it were a standard OpenAI endpoint. The cost is a background process that the user must start separately.

The user points Jan at the proxy's address once, and every Saturn service on the network—current and future—appears as available models without reconfiguration.

The proxy pattern extends to any application that supports a custom OpenAI endpoint: Continue, LM Studio, and dozens of other applications accept the same configuration.

4.3.7 Environment Variable Injection

Aider is a terminal-based AI pair programming tool. It reads `OPENAI_BASE_URL` and `OPENAI_API_KEY` from the environment. The `aider-saturn` CLI wrapper discovers Saturn services, selects the best available service by priority, sets both environment variables, and launches Aider as a subprocess with the correct configuration. The user runs one command instead of manually obtaining API keys, setting environment variables, and specifying model names.

Aider runs unmodified. The wrapper handles discovery, credential extraction (including ephemeral keys from beacon TXT records), and model selection. Optional flags filter by capabilities (`--saturn-needs code,chat`), minimum context window (`--saturn-min-context 64000`), and cost preference (`--saturn-prefer-free`). An interactive mode (`--select`) presents a menu of discovered services and models.

4.4 System Walkthrough

Three walkthroughs show the system performing its core tasks: the administrator deploys a beacon, the developer discovers it, and the end user benefits without configuration.

4.4.1 Administrator: Deploying a Beacon

An administrator provisions a Saturn service in two steps. For a local Ollama instance, the built-in configuration suffices—the administrator runs one command:

```
$ saturn run ollama
INFO:     Saturn beacon registered: ollama._saturn._tcp.local.
INFO:     Models: llama3.2:latest, qwen2.5-coder:latest
INFO:     Endpoint: http://192.168.8.1:11434/v1
INFO:     Priority: 50 | Auth: none | API: ollama
INFO:     Press Ctrl+C to stop
```

Figure 4.1: Terminal output from `saturn run ollama`. The beacon registers on mDNS and advertises the backend's models, endpoint, and metadata.

The administrator verifies network visibility with `saturn discover`:

```
$ saturn discover
ollama (priority: 50)
  Deployment: network | API: ollama
  Endpoint:   http://192.168.8.1:11434/v1
  Models:     llama3.2:latest, qwen2.5-coder:latest
  Capabilities: chat, code
  Context:    128000 | Cost: free
```

Figure 4.2: Output from `saturn discover`, confirming the service is visible via mDNS.

No further configuration is needed—every device on the network receives the beacon automatically **A4**.

4.4.2 Developer: Discovering and Using a Service

A developer integrating Saturn calls two functions:

The developer imports two functions, discovers services on the network, selects the best match by capability, and receives a resolved endpoint ready for HTTP requests **A3**.

4.4.3 End User: AI Without Configuration

An end user encounters Saturn through an application that has already integrated it. In VLC, the user activates the extension through View → Extensions → Saturn Chat.

```
>>> from saturn import discover, select_best_service
>>> services = discover(timeout=5.0)
>>> len(services)
2
>>> service = select_best_service(services, needs=["chat"])
>>> service.name
'ollama'
>>> service.effective_endpoint
'http://192.168.8.1:11434/v1'
```

Figure 4.3: Python session demonstrating Saturn's two-function discovery API. The call resolves endpoint, authentication, and model list from mDNS at runtime.

A dialog appears with a service dropdown populated from mDNS discovery, a model selector, and a chat input. The user types a question about the currently playing media; the extension extracts metadata—title, artist, album, playback timestamp—from VLC's input item API and sends both the question and the context to a Saturn-discovered endpoint. The response appears in the dialog. From the user's perspective, AI capability is a feature of the application, not a service they configured **A3**.

Chapter 5 evaluates whether these nine components—built on four mDNS libraries in four languages, integrated through six patterns—interoperate on the shared wire format defined in Chapter 3.

Chapter 5

Evaluation

5.1 Evaluation Approach

Saturn advances three claims. Claim 1 **V1**: zero-configuration AI discovery is feasible via mDNS/DNS-SD. Claim 2 **V2**: network-level provisioning reduces total configuration effort. Claim 3 **V3**: the security trade-offs of broadcast-based discovery are documented and addressable. Each claim requires a different measurement strategy.

Claims 1 and 2 concern behavioral properties — what systems and users do. We evaluate Claim 1 through requirements decomposition and protocol analysis. We evaluate Claim 2 through cognitive walkthroughs [42] that count configuration steps across three personas. Claim 3 concerns information flow properties — what the network exposes. We evaluate it through STRIDE threat modeling, exposure window analysis, and an honest assessment of the deployment constraints that shape real-world viability.

The evaluation that follows is analytical, not experimental. An earlier automated evaluation framework — in which Claude Code subagents acted as blind test participants in Docker sandboxes — was removed from the thesis after the author could not explain or defend the methodology in a meeting with the thesis advisor. The subagent-

designed tests, grading scripts, and statistical procedures existed as artifacts of agent output rather than as work the author understood and owned. This is documented in full in Appendix B. The evaluation below uses methods the author can explain, defend, and reproduce: requirements analysis, cognitive walkthroughs with executable scripts, and protocol-level security analysis.

5.1.1 Requirements Decomposition

Claim 1 decomposes into three testable requirements:

R1: Service Discovery. Saturn services can be found on the local network without endpoint URLs, API keys, or configuration files on the client side.

R2: Connection Sufficiency. The information carried in mDNS/DNS-SD records provides everything a client needs to establish a working API connection, with no out-of-band configuration.

R3: Cross-Implementation Interoperability. Independent implementations in different languages consume the same protocol records without a bridging or adaptation layer.

R1 asks the simplest version of Claim 1: can a client with no prior knowledge find and connect? R2 tests whether discovery provides enough information for a complete connection — every concern (address, dialect, credentials, routing) must map to a protocol field, or the system still depends on side channels. R3 checks whether independent codebases in different languages consume the same records without shared code.

5.2 Discovery Feasibility (Claim 1)

5.2.1 R1: Service Discovery

Saturn registers services under `_saturn._tcp.local.` and clients discover them through the resolution sequence defined in Section 3.5.4 **A3**.

Six independent implementations confirm this works in practice (Table 4.1). Each follows the same sequence—browse PTR, resolve SRV, parse TXT—using its own mDNS library with no shared Saturn-specific discovery code.

Saturn satisfies the zero-configuration criterion defined in Section 2.2: a device joins the network, multicasts a query, and receives working AI endpoints without prior setup.

5.2.2 R2: Connection Sufficiency

Knowing a service exists is not the same as being able to use it. A client still needs a network address, an API dialect, the right endpoint path, and a credential. If any of those travel through a README, a `.env` file, or a Slack message from IT, the system is not zero-configuration. Connection sufficiency tests whether the zero-configuration standard defined in Section 2.2 survives past discovery into actual use.

Clients follow the resolution sequence defined in Section 3.5.4. The connection-sufficiency question is whether those records carry everything the client needs. Table 5.1 maps each connection concern to its DNS record. No row requires out-of-band information.

For cloud backends, the TXT record carries a live API key generated by Saturn and rotated every five minutes. Clients read it at discovery time and pass it as a Bearer token—the credential arrives through the same multicast channel as the endpoint ad-

Table 5.1: Connection concerns mapped to DNS-SD records.

Connection Concern	Record	Field(s)	Out-of-Band?
Service existence	PTR	instance name	No
Host and port	SRV	target, port	No
IP address	A/AAAA	address	No
API endpoint URL	TXT	`api_base`	No
API compatibility	TXT	`api_type`	No
Authentication	TXT	`ephemeral_key`, `auth`	No
Routing priority	TXT	`priority`	No

dress, eliminating the out-of-band provisioning that Section 2.1 documented.

Table 5.2 shows the schema across three backends with different authentication mechanisms. None required modification to Saturn's TXT schema.

Table 5.2: API backends operating through Saturn's TXT record schema.

Backend	api_type	Auth Mechanism	Deployment	Confirmed In
Ollama	`ollama`	None (local)	Network	Core, Router, AI SDK
DeepInfra	`openai`	Scoped JWT	Cloud beacon	Core
OpenRouter	`openai`	Ephemeral key	Cloud beacon	Core, Router

Ollama skips authentication entirely: `auth` reads `none` and `ephemeral_key` stays empty. DeepInfra uses scoped JWTs that expire on their own. OpenRouter requires explicit key provisioning and revocation. Despite these different mechanisms, the client code path is identical: read TXT, extract any key, authenticate, call. The per-application credential burden documented in Section 2.1 disappears: Saturn's TXT schema replaces it with a single network broadcast.

Limitation. The TXT record schema describes how to *connect* to a service but not what the service *can do* at fine granularity. The optional `capabilities` field advertises broad categories (chat, code, vision), but per-model attributes — context window size, tool-use support, embedding dimensions — fall outside the discovery record. Clients that need this information query the `/v1/models` endpoint after connecting. This re-

flects a protocol constraint: mDNS TXT records are limited to 255 bytes per key-value pair [10], and per-model metadata would exceed that budget.

5.2.3 R3: Cross-Implementation Interoperability

A protocol that only one codebase can consume is a library wearing a specification. Hidden assumptions, hard-coded paths, shared utility modules — none of these can be ruled out from a single implementation. Seven consumers of `_saturn._tcp.local.` records (Table 5.3) test whether the specification does the work or the reference code does.

Table 5.3: Saturn implementation matrix. Seven consumers span three languages, four mDNS libraries, and four integration patterns.

Component	Language	mDNS Library	Integration
Core Python package	Python	`zeroconf`	Native SDK
saturn-router	Rust	`mdns-sd`	Native SDK
ai-sdk-provider-saturn	TypeScript	`multicast-dns`	Native SDK
Open WebUI plugin	Python	`zeroconf`	Application plugin
saturn-mcp	Python	`zeroconf` (via core)	Tool protocol bridge
VLC extension	Lua/Python	`dns-sd` CLI	Subprocess bridge
OpenCode-Saturn	TypeScript	`multicast-dns` (via ai-sdk)	Application fork

Four mDNS libraries underpin these implementations (Table 5.4) **A2**. Saturn patches none of them — each implements RFC 6762 and RFC 6763 on its own and resolves `_saturn._tcp.local.` through its own protocol stack.

Table 5.4: mDNS libraries used across Saturn implementations.

Library	Language	Implementation
`zeroconf`	Python	Pure Python multicast DNS
`mdns-sd`	Rust	Pure Rust multicast DNS
`multicast-dns`	JavaScript	Pure JavaScript multicast DNS
`dns-sd` CLI	C (system)	Apple Bonjour / mDNSResponder

Integration varies by what each host application allows — native SDK embedding,

plugin systems, tool protocol bridges, and subprocess bridges (Chapter 4 details each pattern). The relevant fact for interoperability is that all seven consumers resolve the same `_saturn._tcp.local.` records through independent protocol stacks.

Four independent mDNS libraries implementing RFC 6762 resolve one service type without patches—the interoperability evidence rests on the libraries, not on shared Saturn code.

Limitation. All seven implementations share a single author. Independent development teams would strengthen the interoperability claim. However, the four mDNS libraries — `zeroconf`, `mdns-sd`, `multicast-dns`, and Apple's `dns-sd` — are maintained by independent developers with no knowledge of Saturn. The protocol's interoperability rests on these libraries' implementations of mDNS/DNS-SD, not on shared code between Saturn consumers.

5.2.4 Integration Opportunity

R1 through R3 demonstrate that Saturn's seven implementations discover, connect, and interoperate. The question that follows is scope: how broadly does the protocol's design extend beyond the applications its author chose to target? The VLC extension demonstrates this pattern for consumer applications: a user opens a media player, clicks a menu item, and talks to an AI service without encountering the discovery protocol. The integration opportunity analysis tests whether Saturn's reuse of mDNS/DNS-SD aligns with how existing AI applications already handle endpoint configuration.

This analysis measures protocol compatibility—whether an application's existing extensibility mechanisms can support Saturn discovery. It does not measure adoption likelihood or development effort. A compatible application can technically integrate; whether its maintainers choose to is outside the scope.

We surveyed 42 applications across seven categories, assessing each on two criteria: (1) does the application already accept custom OpenAI-compatible endpoints? and (2) does the application support plugins, extensions, or configuration that would enable Saturn discovery?

Table 5.5: Integration targets by category and viability. "Endpoint" means the application already accepts custom OpenAI-compatible API URLs. "Plugin" means the application supports extensions that could perform Saturn discovery. "Proxy" means the application would require an external bridge (like VLC's subprocess pattern).

Category	**Count**	**Primary Integration Path**
AI coding assistants	8	Endpoint configuration or AI SDK
AI chat applications	6	Endpoint configuration
Note-taking / PKM	4	Plugin systems
Home automation	2	Plugin / integration frameworks
Content creation	4	Plugin systems
Workflow automation	3	HTTP node configuration
Media applications	2	Subprocess bridge
Other developer tools	13	Mixed
Total	**42**	

The highest-viability category is coding assistants: most already accept arbitrary OpenAI-compatible endpoints, so Saturn's `ai-sdk-provider-saturn` package drops in as a provider. Chat applications and workflow platforms are similarly HTTP-native. Note-taking and content creation tools require plugin or scripting interfaces. Media applications need a subprocess bridge.

Every surveyed application supports at least one of three integration patterns demonstrated by Saturn's existing implementations: native SDK (17 targets import an mDNS library directly), application plugin (12 targets host discovery code in an extension system), or subprocess bridge (13 targets connect through a local proxy) **A10**. The proxy pattern serves as a universal fallback — any application that accepts a custom endpoint URL can consume Saturn services, even without plugin support or mDNS capability.

Saturn's protocol aligns with how these applications already handle AI integration.

The gap between “accepts a custom endpoint URL” and “discovers endpoints automatically” is the gap Saturn fills. For 17 of the 42 targets, filling that gap requires importing one library and calling one function.

5.3 Configuration Effort (Claim 2)

Saturn centralizes AI configuration on a single administrator **A4**. Developers inherit the result; end users never see it. The savings grow with headcount — every additional user is one fewer credential setup. We measured this with cognitive walkthroughs [42]: step counts across three personas, all targeting the same completion criterion.

5.3.1 Methodology

A cognitive walkthrough [42] traces a user through an interface step by step, checking whether the correct action is identifiable at each point. We adapted it to compare Saturn against traditional API-key provisioning, starting from infrastructure setup and ending at a working AI feature in an end user’s hands.

The completion criterion: *an end user on the network uses an AI-powered feature in an application*. Both paradigms must reach this finish line. Without a shared endpoint, Saturn’s full pipeline gets measured against the traditional paradigm’s first step — unfair in one direction.

Three personas span the provisioning chain:

1. **Sysadmin/IT** — provisions AI access at the infrastructure level.
2. **App developer** — builds an application that consumes AI services.
3. **End user** — uses an app with AI features.

Both paradigms share eight prerequisite steps: sign up with OpenRouter, generate a key, save it to `.env`. They diverge after that. Every walkthrough is an executable Python file in the project repository (`cog_walkthrough/*.py`); every import, function call, and configuration line counts as one step.

5.3.2 Results

Sysadmin Configuration

Table 5.6: Sysadmin configuration steps.

Paradigm	Prereqs	Paradigm-Specific	Total
Traditional	8	4	12
Saturn (package)	8	6	14
Saturn (manual)	8	18	26

After the prerequisites, a traditional sysadmin emails the key, communicates the endpoint URL, and repeats key creation per project — four steps. Saturn asks for more: install `saturn-ai`, configure a `ServiceConfig` object, call `run_service()`. Six steps with the package; eighteen from scratch.

That is 17% more sysadmin work (117% without the package). But the output differs. A traditional sysadmin hands off a raw key; every downstream developer configures it alone. A Saturn sysadmin leaves a running service on the network, discoverable by every device that joins.

App Developer Configuration

Table 5.7: App developer configuration steps.

Paradigm	AI Integration	Billing Integration	Total
Traditional	6	13	19
Saturn	4	0	4

Traditional integration: install the SDK, import modules, receive a key from IT, store it in an environment variable, instantiate a client, test — six steps. Then thirteen more for billing: a Stripe account, pricing tiers, the Stripe SDK, checkout and webhook endpoints, subscription gating, payment testing. The billing code exists because the developer passes API costs to end users.

Saturn cuts that to four: install `saturn-ai` and the OpenAI SDK, import discovery, call `discover()`, point a client at the result. Billing vanishes because the sysadmin absorbs the API cost at the network level.

End User Configuration

Table 5.8: End user additional steps (beyond shared app onboarding).

Paradigm	Additional Steps	Nature
Traditional	7	Payment friction
Saturn	0	No paywall

Both paradigms share download, account creation, and login. After that they diverge at the paywall. A traditional end user navigates pricing, compares tiers, enters a credit card, accepts terms, confirms, and returns to the AI feature — seven steps. With Saturn there is no paywall; the developer never built billing. AI features work on first launch.

5.3.3 Analysis

Two extra steps for the sysadmin; fifteen fewer per developer; seven fewer per end user **A11**. At 1+1+1 scale the total drops 53%.

Scale the arithmetic. Traditional provisioning costs $12 + 19N + 7M$ steps for N developers and M end users. Saturn costs $14 + 4N + 0M$. At N=10 developers and M=100 users: 902 steps the traditional way, 54 with Saturn — a 94% reduction.

Table 5.9: Configuration steps by persona (Saturn package vs. traditional).

Persona	Traditional	Saturn	Change
Sysadmin	12	14	+2 (+17%)
App Developer	19	4	−15 (−79%)
End User	7	0	−7 (−100%)
Total (1+1+1)	38	18	−20 (−53%)

The walkthrough results connect to the literature. Key leakage [23] no longer scales with developer count because developers never handle keys. Payment friction [35]—the seven end-user steps—disappears entirely. Costa et al. [11] framed usability as a dimension of AI democratization; Saturn's step reductions materialize that framing.

Limitations. These walkthroughs were authored and counted by a single researcher. The step counts are analytical — derived from executable walkthrough scripts in `cog_walkthrough/*.py` — rather than empirical measurements of time or error rates with human participants. A multi-evaluator study with timed task completion would strengthen external validity. The walkthrough scripts are included in the project repository for reproducibility.

5.4 Usability Heuristic Evaluation

Counting steps measures effort, not quality. Nielsen's ten usability heuristics [25], distilled from factor analysis of 249 usability problems, evaluate what the walkthrough cannot: whether the interface itself helps or hinders.

5.4.1 Method

All ten heuristics were evaluated against Saturn's three personas — sysadmin, developer, end user — checking each heuristic against the interfaces that persona encounters. Results fall into three bins: where Saturn helps, where it introduces friction, and where

it fails.

5.4.2 Strengths

Four heuristics favor Saturn.

Table 5.10: Usability heuristics where Saturn demonstrates strength.

ID	Principle	Saturn Evidence
H1	Visibility of system status	Servers appear on network via mDNS; logs show registration of addresses, ports, and service names
H5	Error prevention	Auto-assigns host, port, priority, and failover; defaults to alternate services before erroring
H6	Recognition over recall	`discover()` returns available services; replaces 15 recall-dependent developer steps
H7	Flexibility and efficiency	Python package for programmatic control; traditional API key bypass remains available

`discover()` is the mechanism behind the 19-to-4 developer step reduction. It replaces recall-dependent tasks — typing URLs, locating keys, exporting environment variables — with a function that shows what the network has. Nielsen [25] called this recognition over recall (H6); the network itself serves as the retrieval cue.

Because nobody types a URL, nobody mistypes one. Auto-assigned hosts, ports, priorities, and failover targets eliminate what Nielsen [25] classified as "slips" — unconscious errors from inattention (H5).

5.4.3 Trade-offs

Three heuristics cut both ways — Saturn's choices help one persona at the expense of another.

Table 5.11: Usability heuristics where Saturn presents trade-offs.

ID	Principle	Saturn Finding
H2	Match real-world conventions	"Beacon" and "proxy" confuse sysadmins; developers and end users less affected
H3	User control and freedom	Developers pick servers by name; sysadmins stop services with ctrl-c; end users have no service-level control
H4	Consistency and standards	TXT schema drifts across implementations: `auth` field TypeScript-only; `capabilities`, `cost`, `context` missing from Rust

H4 (consistency) qualifies R3. Seven consumers interoperate on `_saturn._tcp.local.`, but agreement on the service type does not guarantee agreement on the schema. Four core fields — `version`, `deployment`, `api_type`, `priority` — appear in every implementation. Beyond those, drift: TypeScript carries an `auth` field absent from Python and Rust; Rust omits `capabilities`, `cost`, and `context`. Discovery succeeds, but the developer experience in mixed deployments degrades.

H2 (real-world match) falls on the sysadmin. "Beacon" accurately describes Saturn's architecture — but a sysadmin hears "wireless access point." "Proxy" in the codebase means API relay, not the HTTP/SOCKS proxy a sysadmin expects. Developers interact with `discover()` and rarely encounter either term. End users never do.

5.4.4 Limitations

Three heuristics expose gaps.

H9 (error recovery) is the clearest failure. When `discover()` returns nothing, the message reads "service not available — check the network." Is the server down? Is multicast blocked? Is a firewall dropping UDP 5353? No clue. Nielsen [25] required that error messages name the problem and suggest a fix; Saturn's do neither. A diag-

Table 5.12: Usability heuristics where Saturn shows limitations.

ID	Principle	Saturn Gap
H8	Minimalist design	TXT schema inconsistency adds noise; `_saturn._tcp.local.` is specialist-facing
H9	Error recovery	Only "service not available" and "check network" — no diagnostic guidance
H10	Help and documentation	Docs cover sysadmin and developer audiences; end-user documentation is thin

nostic mode — multicast reachability check, nearby-service scan, targeted suggestions — would close the gap.

H8 (minimalist design) amplifies H4. In mixed deployments, different implementations advertise different TXT schemas for the same service type — noise where minimalism demands consistency. The service name `_saturn._tcp.local.` is opaque to anyone outside an mDNS debugger, though in practice only sysadmins encounter it.

5.4.5 Synthesis

H6 and H5 explain the 79% developer step reduction: recognition replaces recall; auto-configuration prevents slips. The costs land on the sysadmin, who faces confusing terminology (H2) and a drifting schema (H4). H9 — error recovery — remains the largest open problem. Saturn offers no self-diagnosis.

The walkthrough counted steps; the heuristics judged the interface. Both are single-evaluator inspections, and empirical user studies would strengthen external validity.

5.5 Security Analysis (Claim 3)

Traditional API key distribution happens out-of-band — credentials live in `.env` files, environment variables, and secret management systems. Nothing leaks to the network.

Saturn puts metadata on the wire **A5**. The analysis below characterizes what that costs.

Because the exposure surface is defined by the TXT record schema (Appendix A), not by runtime behavior, the analysis is protocol-level: what the specification requires every Saturn beacon to broadcast, compared against static API keys and OAuth through the STRIDE framework.

5.5.1 Information Leakage Surface

Eleven metadata fields go out in every Saturn beacon's TXT records (Table 5.13). The protocol specification defines all of them.

Table 5.13: Fields exposed by Saturn protocol design.

TXT Record Field	Content
`api_base`	Endpoint URL (e.g., `http://host:8001/v1`)
`api_type`	API compatibility type (e.g., `openai`)
`deployment`	Deployment context (`local` or `cloud`)
`priority`	Routing priority value
`version`	Protocol version
`capabilities`	Supported modalities (e.g., `chat`)
`context`	Context window size
`cost`	Cost indicator
`features`	Optional feature flags
`models`	Available model identifiers
`service_type`	PTR record for service enumeration
11 fields exposed. 0 credential material in base configuration.	

An observer on a Saturn network learns which AI services exist, where they run, which models they offer, and how they rank. Local deployments expose no credentials. Cloud deployments add an ephemeral key with the lifecycle defined in Section 3.4.4. Prompts and responses remain invisible — beacons announce where services live; actual API traffic flows over HTTPS between client and endpoint.

5.5.2 STRIDE Threat Comparison

Table 5.14 compares three credential approaches across STRIDE's six threat categories.

Table 5.14: STRIDE severity by credential approach.

Threat Category	**Static Keys**	**OAuth/SSO**	**Saturn**
Spoofing	High	Low	Medium
Tampering	Low	Low	Medium
Repudiation	High	Low	Medium
Information Disclosure	High	Medium	Low
Denial of Service	Low	Low	Medium
Elevation of Privilege	High	Low	Low
High / Medium / Low	3/0/3	0/1/5	0/3/3

Static keys carry three high-severity threats. A bearer token equates possession with identity (spoofing). Shared keys provide no per-user attribution (repudiation). A stolen admin-scoped key grants full API access (elevation of privilege). Saturn eliminates all three high-severity categories **A6**. Spoofing drops to medium because mDNS announcements are spoofable on the local segment, but 10-minute key expiry limits the impersonation window. Information disclosure drops to low because the credential expires automatically — static keys, by contrast, persist until manually revoked (Section 2.1.3).

The trade-off is geographic scope. Saturn's three medium-severity threats — spoofing, unsigned records, multicast flooding — require physical presence on the LAN. Static key exploits work from anywhere on the internet.

5.5.3 Exposure Window Analysis

The exposure window — how long a stolen credential remains dangerous — is where Saturn's ephemeral design pays off.

At 10 requests/second and $0.01/request, a captured Saturn key caps damage at $60

Table 5.15: Credential exposure window comparison.

Metric	Static Key	OAuth Token	Saturn Key
Max lifetime	Unbounded	1–24 hours	10 minutes
Rotation	Manual	Refresh flow	Automatic (5 min)
Network scope	Global	Global	Local segment
Revocation	Manual (rarely done)	Automatic	Automatic

— ten minutes of traffic, then the credential expires. A static key unnoticed for one day: $8,640. Because static keys rarely get revoked (Section 2.1.3), their exposure has no upper bound.

Saturn's automatic expiry removes revocation from the equation. The administrator does not need to detect the leak, find the compromised key, and rotate it. The key dies on schedule regardless.

5.5.4 AP Isolation and Deployment Constraints

Saturn depends on multicast reachability **A17**. Enterprise WiFi routinely blocks it.

In January 2026 we tested on two campus networks. Eduroam and UCSC-guest both silenced mDNS — clients multicast to `224.0.0.251:5353` and hear nothing back. The campus scenario from Chapter 1 fails on campus. Universities, libraries, co-working spaces — the environments where Saturn's access argument is strongest — are the ones most likely to run AP isolation.

No protocol-level workaround exists. Campus IT decides whether multicast traffic flows; Saturn cannot route around that decision without network changes (enabling multicast forwarding, deploying on a dedicated segment). VLANs without multicast bridging and firewalls blocking UDP 5353 produce the same result.

Where the administrator controls the network and multicast flows freely — home WiFi, small offices, lab networks, wired Ethernet — Saturn works. Where client isolation blocks multicast, it does not. That constraint belongs to mDNS itself (RFC 6762 [10]),

not to Saturn. The protocol was designed for link-local scope; institutional networks that suppress multicast between clients fall outside that scope by administrative choice.

5.5.5 Security Posture Across Implementations

Saturn's modular architecture means different components adopt different security postures, ranging from full ephemeral lifecycle to none at all.

The `saturn-router` Rust implementation runs the complete ephemeral key lifecycle: generate, broadcast, rotate every five minutes, delete expired keys, and auto-deregister unhealthy services. The core Python package's `KeyManager` implements the same rotation for OpenRouter beacons. The TypeScript `ai-sdk-provider-saturn` adds a circuit breaker (three failures trip the breaker, 30-second cooldown) and automatic key refresh on HTTP 401 responses.

At the other end, `owui-saturn` (the Open WebUI plugin) contains no authentication logic. Its security boundary is upstream — Open WebUI's own user management handles access control. The VLC bridge binds to localhost and has an unauthenticated `/shutdown` endpoint.

This gradient is deliberate. A home router serving a family network has different security requirements than a cloud beacon distributing ephemeral keys to a campus lab. Saturn's protocol defines what can be broadcast; each implementation decides what security measures to layer on top.

5.6 Threats to Validity

The cognitive walkthrough step counts are analytical, not empirical. They were produced by a single researcher tracing through executable Python scripts, not by timing multiple human participants completing real tasks. The scripts are reproducible — an-

other researcher can run the same code and arrive at the same step counts — but the mapping from "steps in a script" to "cognitive effort experienced by a human" is imprecise. A multi-evaluator study with timed task completion and error rate measurement would strengthen external validity.

The heuristic evaluation was conducted by a single evaluator. Nielsen [25] recommended three to five evaluators to achieve adequate coverage. A single evaluator typically finds 35% of usability problems. The findings reported here should be treated as a lower bound.

All seven Saturn implementations share a single author. The interoperability claim rests on the independence of the four mDNS libraries (maintained by developers with no knowledge of Saturn), not on the independence of the Saturn consumers themselves. An implementation by an unaffiliated developer would provide stronger evidence for R3.

Saturn relies on multicast, which does not route across subnets. All evaluation occurred on single-segment networks (home WiFi, lab LAN). Saturn targets this deployment model (home networks, small offices, dorm networks); multi-subnet discovery is future work.

The security analysis characterizes what the protocol exposes by design; it does not simulate adversarial attacks. Active penetration testing — DNS rebinding, ARP spoofing, rogue service registration, key exfiltration — would reveal practical exploitability beyond passive analysis. Detailed threat scenarios appear in Chapter 6.

Chapter 6

Discussion

The evaluation established three results: Saturn discovers AI services without end-user configuration (**V1**), concentrates setup work in one administrator while eliminating it for everyone else (**V2**), and exposes a bounded, documented set of security trade-offs (**V3**). This chapter interprets those results—what they mean for the networks Saturn targets, where the design deliberately diverges from established security practice, what the work cannot claim, and where it points next.

6.1 Threat Model

Saturn trades enterprise-grade verification for zero-configuration access. That trade-off plays out differently depending on who controls the network. Four scenarios span the range.

6.1.1 Home Network

The administrator and the user are the same person. The threat surface is minimal: no untrusted parties share the segment, multicast traffic stays within the household,

and the administrator who runs `saturn run` controls every device that receives the beacon. AP isolation is off by default on consumer routers. This is Saturn's strongest deployment scenario—the environment closest to DHCP's own trust model, where the entity assigning addresses also controls physical access.

6.1.2 Campus Lab or Classroom

A teaching assistant or lab manager runs Saturn on a dedicated subnet. Students on that subnet discover services without accounts. The administrator controls who joins the network (via WiFi credentials) and what services appear (via Saturn's TOML configuration). Multicast works because lab networks typically lack client isolation.

The residual risk is a student on the same subnet broadcasting a rogue `_saturn._tcp.local.` service. mDNS does not authenticate service registrars—a vulnerability inherent to the protocol (Section 2.5.2). Saturn mitigates it through administrator-set priority values: legitimate beacons advertise low priority numbers (e.g., 10) and rank above any rogue service using default values (**A8**). This raises the bar above trivial spoofing but does not prevent a determined attacker on the same segment.

6.1.3 Coffee Shop

An open WiFi network with no administrative control over who joins. Any device on the segment can register mDNS services and advertise itself as an AI provider. A malicious actor could broadcast a `_saturn._tcp.local.` service pointing to an endpoint that captures prompts or returns manipulated responses.

Saturn is not appropriate for this environment. The design targets administratively controlled networks where the set of legitimate service providers is managed (**D1**). A coffee shop owner who wanted to offer Saturn would need a dedicated SSID with

client isolation disabled and a beacon running on trusted hardware—at which point the scenario collapses back to the lab case.

6.1.4 Compromised Administrator

The administrator who runs Saturn controls what services appear, what priority values are assigned, and—for cloud backends—what API keys are distributed. A malicious administrator could advertise a rogue endpoint, capture prompts, or redirect traffic to a logging proxy.

This threat is real but not novel. A malicious network administrator already controls DNS, DHCP, firewall rules, and routing tables. Saturn does not introduce new capabilities for compromise; it inherits the standard LAN trust model. The critical design decision is that beacons do not proxy traffic (**D3**): actual API communication uses HTTPS between client and endpoint. The administrator cannot read encrypted traffic without also compromising the endpoint's TLS certificate.

Logging is the residual risk. An administrator running a local Ollama instance can access Ollama's request logs. This is a property of the backend, not of Saturn.

6.2 The BeyondCorp Divergence

The BeyondCorp model (Section 2.5) requires managed devices, identity providers, and per-request verification. Saturn deliberately inverts these principles (**AV1**).

The divergence is not accidental. BeyondCorp requires managed devices, an identity provider, and an access control engine—infrastructure that campus WiFi users, coffee shop patrons, and home network users do not have. The overhead of per-request verification destroys zero-configuration: a system that asks "who are you?" before every API call is not zero-config, regardless of how the credential was provisioned. Sat-

urn's target users—students without IT departments, researchers with a personal laptop, teachers setting up a classroom—fall outside BeyondCorp's operational assumptions.

The cost is real. Saturn cannot verify that a discovered service is legitimate, cannot authenticate the device requesting access, and cannot enforce per-user access policies. These are capabilities BeyondCorp provides. Saturn trades them for a property BeyondCorp does not offer: discovery and connection with no prior enrollment.

Where the two models might converge is Osborn et al.'s tiered trust [33]. A future Saturn deployment could default to open access (current behavior) but support opt-in verification for sensitive environments: device certificates for institutional deployments, PIN-based pairing for shared spaces. Kaiser and Waldvogel's privacy sockets [19] provide a compatible mechanism at the mDNS layer. Section 6.7 develops this direction.

6.3 API Key Leakage: A Comparison

Saturn's ephemeral key design (**A5**) responds to the key persistence problem established in Section 2.1.3. The fundamental issue is temporal—damage occurs before detection, and detection does not trigger remediation. In the per-user key model (Section 2.1), N users managing credentials independently produce N potential leaks [35].

Saturn sidesteps the failure class rather than mitigating it. The ephemeral key lifecycle (Section 3.4.4) ensures a captured key expires before an attacker can build anything on it. Keys never appear in configuration files, environment variables, or source repositories—they exist only in mDNS TXT records, available only to devices currently on the network segment. There is no inventory problem because there is no persistent credential to inventory.

The trade-off is visibility. Static keys live in `.env` files readable only by the local

filesystem. Saturn's ephemeral keys live in multicast DNS, readable by every device on the segment. The exposure window is wider but shorter. For the target environments—home networks, labs, offices where the administrator controls physical access—the ephemeral model exchanges filesystem privacy for temporal containment.

6.4 AP Isolation: The Hard Blocker

Enterprise and institutional WiFi networks block multicast traffic between wireless clients (**A17**, **B5**). Section 5.5.4 confirmed the consequence: on both campus networks tested, discovery failed completely.

This is Saturn's most consequential limitation because it inverts the deployment map. The environments where shared AI provisioning would help most—universities, public libraries, community colleges—are the environments most likely to run enterprise WiFi with client isolation enabled. Bassignana et al. [3] and Carmona-Galindo et al. [7] documented the AI access gap at exactly these institutions.

AP isolation is not solvable within Saturn's protocol. It is a network infrastructure decision made by campus IT departments for defensible security reasons (preventing lateral movement between wireless clients). Saturn works on its target networks—home WiFi, small offices, lab subnets, wired Ethernet segments—where consumer routers forward multicast by default and administrators control the infrastructure (**B5**). Institutional deployment requires either a dedicated SSID with isolation disabled or a wired Ethernet segment where the beacon and clients share a switch.

6.5 Limitations and Generalizability

Beyond AP isolation, several boundaries constrain what this work establishes.

Single network segment. mDNS is link-local by design (RFC 6762 [10]). Saturn does not provide discovery across subnets, VLANs, or wide-area networks without infrastructure changes (**AV2**, **CA2**). This is intentional—the security model depends on the network trust boundary—but it means Saturn cannot serve a distributed campus with multiple Layer 3 segments without running beacons on each segment.

Analytical evaluation. The cognitive walkthrough uses step counts derived from Wharton et al.'s methodology [42], not empirical user timing (**B4**). The 53% total reduction (**A11**) and 79% developer reduction (**A12**) reflect what an informed analyst counts, not what real users experience. Human-subject evaluation would strengthen claims about usability.

Schema inconsistency across implementations. The Nielsen heuristic evaluation (**A18**) identified drift in TXT record fields: the `auth` field exists only in the TypeScript implementation, and the Rust implementation omits `capabilities`, `cost`, and `context` (**B6**). The four core fields (`api_base`, `api_type`, `deployment`, `priority`) are consistent across all implementations, but peripheral fields diverge.

No adversarial testing. The security analysis is analytical—protocol inspection and STRIDE threat comparison, not penetration testing. Active attacks (DNS rebinding, ARP spoofing, rogue service registration with crafted priority values) were not simulated. The threat model describes what is possible; it does not measure what is exploitable.

Single-author system. Saturn is the only implementation of network-level AI provisioning (**B1**). No independent team has deployed the protocol, which means interoperability evidence comes from implementations built by the same developer. The interoperability evidence (Section 5.2.3) rests on four independent mDNS libraries (**A10**), which provides protocol-level independence, but the application-layer integration patterns have not been validated by external developers.

6.6 Relation to Prior Work

6.6.1 Service Discovery

Saturn extends the mDNS service discovery pattern to AI services: the TXT record schema encodes API type, model list, ephemeral credentials, and routing priority within standard DNS-SD fields. No changes to the wire format, registration mechanism, or resolution sequence were required. The contribution is not a new discovery protocol but evidence that an existing one carries AI-specific metadata without adaptation—the constraint on zero-configuration AI provisioning was never the transport layer.

6.6.2 AI Provisioning

The per-user credential model (Section 2.1) requires each user to obtain and rotate credentials independently. Saturn intervenes at the provisioning layer: one administrator configures a beacon, and N users discover services with zero credential management (**A4**). The cognitive walkthrough quantified this asymmetry—at 10 developers and 100 end users, total configuration drops by 94% (**A14**)—but the structural insight matters more than the percentage. Saturn shifts provisioning from a per-user cost that scales linearly to a per-network cost that remains constant regardless of user count.

Saturn does not replace cloud AI services. Local and cloud endpoints appear through the same discovery mechanism, which means the provisioning architecture is independent of where the model runs.

6.6.3 Network Security

Saturn accepts the privacy risks inherent in multicast discovery—the passive enumeration and device-name leakage cataloged in Section 2.5.2—as the cost of zero configuration. The modular security posture (**A6**) lets deployments choose their position on this trade-off: saturn-router runs the full ephemeral key lifecycle; owui-saturn runs none. This gradient distinguishes Saturn from systems that impose a uniform security model regardless of deployment context.

Section 6.2 documented the zero-trust divergence; Section 6.7 identifies the convergence path. The broader implication is that edge networks—home labs, classrooms, small offices—operate under trust assumptions that zero-trust architectures were not designed for and cannot serve without infrastructure those environments lack.

6.7 Future Work

Three directions follow from Saturn's current limitations.

6.7.1 Privacy-Preserving Discovery

Kaiser and Waldvogel [19] demonstrated that privacy-preserving mDNS-SD is feasible without breaking backward compatibility. Their privacy sockets encrypt service metadata so that only paired devices can read it. Applied to Saturn, this would prevent passive enumeration of AI service metadata—model lists, API types, ephemeral credentials—while maintaining zero-configuration discovery for authorized users. The pairing step adds one interaction (a PIN or QR code exchange) but preserves the property that no accounts, no cloud services, and no configuration files are required.

6.7.2 Tiered Trust

Osborn et al.'s BeyondCorp tiered trust model [33] assigns access levels based on device posture. A Saturn deployment could implement three tiers: open (current behavior, any device on the segment discovers all services), verified (device presents a certificate or PIN before receiving ephemeral keys), and managed (full identity verification for sensitive deployments). The open tier preserves zero-configuration for home and lab networks. The verified tier addresses coffee shop and shared-space scenarios. The managed tier bridges toward enterprise requirements. Each tier adds configuration overhead in proportion to the security it provides.

6.7.3 Wide-Area Discovery

mDNS is link-local by design (**CA2**). A relay service could forward Saturn beacons across subnets, enabling discovery on multi-segment campus networks. DNS-SD already supports wide-area operation over unicast DNS (RFC 6763 [9]); a campus DNS server could host Saturn service records alongside conventional entries. This approach preserves the protocol but shifts trust: a DNS administrator now curates which Saturn services are visible campus-wide, reintroducing the centralized authority that zero-configuration avoids. The design space involves balancing discovery scope against the trust assumptions that make broadcast-based security tractable.

6.8 Broader Implications

Saturn's core finding is that mDNS/DNS-SD carries AI service metadata without modification. The TXT record schema fits. The discovery latency is acceptable. The security trade-offs, while real, are bounded and documented. This suggests that the barrier to

network-level AI provisioning is not technical but organizational: the protocols exist, the implementations work, and the remaining obstacle is convincing network administrators that AI belongs in the same category as printing.

Bassignana et al. [3] documented the AI gap—socioeconomic status predicting who benefits from language technology. Gabriel and Bossens [14] traced the same dynamic through classrooms, and Capraro et al. [6] across national economies. Carmona-Galindo et al. [7] measured it at Hispanic-Serving Institutions where budgets constrain AI tool availability. Saturn does not solve the global AI access problem. Within a single network segment, however, it enables any administrator to deploy AI services that every device discovers and uses. The cost drops from "each user subscribes independently" to "one administrator runs a daemon." This matters most at resource-constrained institutions—community colleges, public schools, libraries—where every dollar of IT budget counts and where the students who need AI tools most are least likely to have their own subscriptions.

Chapter 7

Conclusion

Saturn demonstrates that AI services can be provisioned like printers, DHCP addresses, and file shares—announced on the local network and discovered without user configuration. The protocol required no changes to mDNS/DNS-SD; the contribution is evidence that existing zero-configuration infrastructure carries AI-specific metadata without adaptation.

Independent implementations interoperate on a shared service type because the protocol specification, not any reference implementation, defines conformance (**V1**). Network-level provisioning shifts the cost structure: one administrator configures a beacon, and every user on the segment discovers services without individual setup (**V2**). The security model trades broadcast exposure for temporal containment—ephemeral keys expire before leaked credentials become useful, at the cost of local-segment visibility that static keys avoid (**V3**).

The structural insight is the asymmetry. Per-user provisioning scales linearly: each person added to a network repeats the same credential acquisition, endpoint configuration, and billing setup. Saturn makes provisioning a per-network constant—one daemon, configured once, serving every device that joins the segment.

Two constraints bound these findings. AP isolation blocks multicast between wireless clients, and the institutions where shared AI provisioning would help most—universities, public libraries, community colleges—are the institutions most likely to run that configuration (**A17**). This is the cost of the design's central choice: zero-configuration requires multicast, and multicast requires cooperative infrastructure. Second, all implementations originate from one developer; independent adoption would strengthen the interoperability evidence.

Wide-area DNS-SD and relay architectures can extend discovery across subnets, but each reintroduces centralized authority. The open question is where that boundary falls—how much coordination overhead can return before the zero-configuration property ceases to hold.

The protocols exist and the implementations interoperate. The remaining barriers are both technical—AP isolation, schema drift, single-segment scope—and organizational—convincing network administrators that AI belongs alongside the services they already provision. Saturn establishes that the technical path is open.

Appendix A

Saturn Protocol Specification

This appendix specifies the Saturn protocol as a standalone reference. An implementor reading only this appendix and the RFCs it cites should produce a system interoperable with any conforming Saturn deployment.

A.1 Service Registration

A Saturn beacon registers one DNS-SD service instance under the type `_saturn._tcp.local.` (RFC 6763 [9]). Registration creates three DNS records:

1. A **PTR record** mapping `_saturn._tcp.local.` to the instance name (e.g., `ollama-lab._saturn._tcp.local.`).
2. An **SRV record** providing the beacon's hostname and port.
3. A **TXT record** carrying service metadata as key-value pairs.

The instance name is a human-readable UTF-8 string chosen by the administrator. It MUST be unique within the multicast domain. The `.local.` suffix confines all queries and responses to the link-local scope defined by mDNS (RFC 6762 [10]).

A beacon MUST register exactly one service instance per advertised endpoint. A host running two backends (e.g., Ollama on port 11434 and a cloud proxy on port 8001) registers two instances with distinct names.

A.2 TXT Record Schema

Table A.1 defines all TXT record fields. One DNS TXT string encodes one key-value pair in `key=value` format and MUST fit within 255 bytes (RFC 6763, Section 6.1 [9]). Clients MUST ignore unrecognized keys to allow forward-compatible schema extensions.

A.3 Endpoint Requirements

Every URL advertised in `api_base` MUST implement three HTTP paths:

`GET /v1/health` Returns JSON with a `status` field. Clients call this to verify liveness before use and periodically during sessions. A non-200 response or timeout (recommended: 5 seconds) indicates the service is unavailable.

`GET /v1/models` Returns a JSON object with a `data` array of model objects, each containing at minimum an `id` field. This follows the OpenAI list-models response format.

`POST /v1/chat/completions` Accepts a JSON body with `model` (string) and `messages` (array of {`role, content`} objects). Returns a chat completion response. MUST support both non-streaming (JSON response) and streaming (server-sent events with `data:` prefix) modes, selected by the `stream` boolean in the request body.

For cloud beacons, `api_base` points to the upstream provider, not the beacon. Clients authenticate with the provider using the `ephemeral_key` from the TXT record.

For local beacons (e.g., Ollama [27]), `api_base` points to the inference server on the local network. No authentication header is required unless `features` includes `ephemeral_auth`.

A.4 Discovery Algorithm

A conforming client discovers Saturn services through the following procedure.

```
function discover():
    instances = mdns_browse("_saturn._tcp.local.")
    services = []
    for instance in instances:
        srv = mdns_resolve_srv(instance)
        txt = mdns_resolve_txt(instance)
        if txt["version"].major != 1:
            skip  // incompatible protocol version
        services.append({
            host: srv.hostname,
            port: srv.port,
            metadata: txt
        })
    sort services by metadata["priority"] ascending
    for service in services:
        url = service.metadata["api_base"]
        headers = {}
```

```
        if "ephemeral_auth" in service.metadata["features"]:
            headers["Authorization"] =
                "Bearer " + service.metadata["ephemeral_key"]
        response = http_get(url + "/v1/health",
                            headers, timeout=5s)
        if response.status == 200:
            return service  // connected
    return null  // no healthy service found
```

Clients MAY cache discovery results. A recommended cache TTL is 60 seconds; clients SHOULD re-browse when the cached service fails a health check.

Clients SHOULD implement fallback: when the current service becomes unreachable, re-execute the discovery algorithm and connect to the next-priority healthy service.

A.5 Ephemeral Key Rotation Protocol

Cloud beacons rotate API credentials on a fixed schedule to bound credential exposure windows. The rotation proceeds as follows:

1. At startup, the beacon creates a new API key k_0 via the provider's provisioning API and records its creation time t_0.

2. The beacon publishes k_0 in the `ephemeral_key` TXT field and sets `rotation_interval` to the rotation period R (default: 300 seconds).

3. At $t_0 + R$, the beacon creates a new key k_1, updates the TXT record to advertise k_1, and starts a grace timer for k_0.

4. After a grace period G (default: 5 seconds), the beacon deletes k_0 from the provider's system via the provisioning API.

5. Steps 3–4 repeat every R seconds for keys $k_2, k_3, \ldots$

6. On shutdown, the beacon MUST delete the current key from the provider.

The key lifetime L (default: 600 seconds) exceeds the rotation interval R, so at most two keys are valid simultaneously during the overlap window. This guarantees zero-downtime transitions: clients using the previous key continue working while the TXT record advertises its replacement.

The grace period G accommodates mDNS propagation delay. During the G-second window after rotation, both old and new keys are valid at the provider. After G elapses, the old key is revoked and any client still using it receives an authentication error, triggering re-discovery.

A.6 Conformance Levels

A.6.1 Beacon Conformance

A conforming Saturn beacon MUST:

- Register exactly one `_saturn._tcp.local.` service instance per advertised endpoint.

- Include all six REQUIRED TXT fields (Table A.1).

- Include `ephemeral_key` and `rotation_interval` when `features` contains `ephemeral_auth`.

- Respond to mDNS queries for its registered instance.

- Deregister the mDNS service on shutdown.
- Delete active ephemeral keys from the provider on shutdown.

A conforming Saturn beacon SHOULD:

- Populate OPTIONAL TXT fields (`models`, `capabilities`, `context`, `cost`) to assist client selection.
- Log key rotation events for administrative auditing.
- Warn if an ephemeral key exceeds 240 characters (approaching the 255-byte TXT string limit).

A.6.2 Client Conformance

A conforming Saturn client MUST:

- Discover services via mDNS PTR query for `_saturn._tcp.local.`.
- Parse all REQUIRED TXT fields; ignore unrecognized fields.
- Sort discovered services by `priority` ascending.
- Verify service health via `GET /v1/health` before use.
- Include an `Authorization: Bearer` header when `features` contains `ephemeral_auth`.
- Fall back to the next-priority service when the current service becomes unreachable.

A conforming Saturn client SHOULD:

- Cache discovery results with a TTL of 60 seconds.

- Re-browse on health check failure rather than waiting for cache expiry.
- Filter services by `deployment` type when client policy requires it.
- Display service metadata (deployment type, model list, cost tier) to users before routing requests.

A conforming Saturn client MAY:

- Maintain a whitelist of trusted service providers.
- Require user confirmation before connecting to previously unseen services.
- Filter by `capabilities` or `models` to select services matching application requirements.

Table A.1: Complete TXT Record Field Reference

Field	Presence	Type	Semantics
`version`	REQUIRED	String	Protocol version. Currently `1.0`. Clients MUST reject services with unrecognized major versions.
`deployment`	REQUIRED	Enum	`cloud` (remote API provider) or `local` (on-network inference server).
`api_type`	REQUIRED	String	API compatibility identifier: `openai`, `ollama`, or provider-specific values.
`api_base`	REQUIRED	URL	Base URL for API requests. Local example: `http://192.168.1.5:11434/v1`. Cloud example: `https://openrouter.ai/api/v1`.
`priority`	REQUIRED	Integer	Routing preference. Lower values indicate higher priority. Clients MUST sort ascending; ties are broken by implementation.
`features`	REQUIRED	String	Comma-separated feature flags. Defined values: `ephemeral_auth` (key rotation active), `network_proxy` (local proxy service).
`ephemeral_key`	Conditional	String	Temporary API credential. REQUIRED when `features` includes `ephemeral_auth`. MUST fit within 240 usable bytes after key-value encoding overhead.
`rotation_interval`	Conditional	Integer	Key rotation period in seconds. REQUIRED alongside `ephemeral_key`. Default: 300 (5 minutes).
`models`	OPTIONAL	String	Comma-separated model identifiers available at this endpoint.
`capabilities`	OPTIONAL	String	Supported modalities: `chat`, `completion`, `embedding`.
`context`	OPTIONAL	Integer	Maximum context window in tokens.
`cost`	OPTIONAL	String	Pricing tier: `free`, `metered`, or `unknown`.

Appendix B

Generative AI in Thesis Development

Saturn argues AI should be accessible. Hiding our own use of it would undercut the argument. This appendix documents how Claude (Anthropic) supported the thesis development process, where it helped, where it failed, and where human judgment overrode its suggestions.

B.1 Knowledge Graph Authorship

Twenty-odd papers, six codebases, three claims that borrow evidence from each other. By week three a flat document could not track the dependencies. We built a JSON knowledge graph—nodes for concepts, papers, claims, and components; edges typed as `supports`, `contrasts`, or `demonstrated_by`—five hundred lines of structure pointing at linked Markdown files where the actual content lives.

The authorship process had four phases.

Scaffold (October 2025). We defined node types (concept, claim, paper, component, chapter, voice, meta, person) and edge semantics before Claude touched anything. The distinction between `supports` (direct evidence for a claim) and `related` (a con-

nection exists but the paper does not argue our point) mattered more than how fast edges appeared.

Bulk generation (November–December 2025). For each claim, Claude proposed candidate edges and descriptions from paper summaries we provided. It generated roughly a hundred candidates. Some were defensible only with generous reading—Siddiqui et al. [37] as supporting Claim 1 holds if you skip the abstract and find the section on heterogeneous sensor networks, but the link is fragile. We checked every edge against the source text and dropped the ones that stretched.

Refinement (January–February 2026). Consistency passes verified edge directions. Gap detection identified missing connections between concepts and claims. Auto-generated descriptions were rewritten for precision.

Live updates. After each advisor meeting or new insight, we updated both the content file and graph edges. The graph remained a living knowledge base, not a frozen artifact.

The payoff was in cross-domain links. Connecting Kaiser and Waldvogel's mDNS privacy analysis [18] to accessibility literature on AI gaps would have required deliberate searching and a lucky reading list. Claude produced the connection in seconds. We still rewrote most auto-generated descriptions and caught contradictions it missed.

B.2 Iterative Advisory Role

From January 2026 onward, we ran draft sections past Claude for pushback. Two exchanges reshaped the thesis.

Claim 2 originally read "reducing configuration effort." Claude asked: compared to what? We had no baseline. That question forced us to define the status quo—developers managing cloud API keys per-app with no local fallback—and rebuild the evaluation

around the cognitive walkthrough methodology from Wharton et al. [42].

A separate exchange exposed a contradiction we had missed: if mDNS lets anyone broadcast a service, an attacker can broadcast a fake one. We had conflated what Saturn guarantees at the infrastructure level (nothing—mDNS is unauthenticated) with what it mitigates at the application layer (ephemeral keys, priority-based selection). Untangling the two rewired the threat model in Chapter 6.

A framing suggestion—introduce the problem as a forced choice between manual setup and expensive cloud APIs—made it into the final draft in modified form.

Four suggestions did not survive review:

- A fourth claim about cost savings would have diluted the access-equity focus.
- Monte Carlo scalability predictions required parameter estimates we could not ground. We ran real trials on a physical router instead.
- Quantum computing threats gave way to classical attack vectors that were actually relevant to the deployment context.
- A DRM-based model access control scheme was wrong for Saturn's audience. Rejecting it forced us to articulate why Saturn deliberately skips access control—a gap we had not noticed until Claude proposed filling it.

B.3 Design Fictions as Specification Input

Three fictional characters—Derek (sysadmin), Mira (end user), and Jordan (developer who leaked an API key)—anchored Saturn's design fictions. These were created as speculative narratives for discussion, their traditional purpose in HCI research.

Claude treated these fictions not as speculation but as vision documents that could be decomposed into implementable specifications. When we fed the Derek fiction to

Claude, it reverse-engineered the narrative into protocol requirements: TXT record fields, priority semantics, beacon lifecycle. This was discovered, not planned. No prior work has been found supporting this use of design fictions as LLM specification input.

The three characters mapped directly to Saturn's three audience types (sysadmin, app developer, end user), which drove the cognitive walkthrough personas in the evaluation and the understanding that Derek does more work so Mira does none—the asymmetric complexity trade-off at the core of Claim 2.

B.4 Failed Evaluation: A Cautionary Note

The thesis originally included an automated evaluation where Claude Code subagents acted as blind test participants in Docker sandboxes. Claude designed the tests, wrote the grading scripts, built the sandboxes, ran the experiments, and interpreted the results. When the thesis advisor asked how the evaluation worked, the author could not explain or defend the work.

The evaluation was removed in February 2026. The lesson: producing work you cannot explain is worse than producing no work at all. The claims survived because they are valid independent of how they are evaluated. The replacement evaluation—cognitive walkthroughs, protocol analysis, implementation census—reflects work the author understands and can defend.

This failure is documented because it illustrates a real risk of AI-assisted academic work. The tools accelerate production but do not transfer understanding. Delegating too much produces artifacts rather than scholarship.

B.5 Limits and Human Judgment

Without Claude the thesis would still exist. The graph would have taken longer to build; the Kaiser-to-accessibility link might never have occurred to us; the Claim 2 baseline question might have survived another draft. Design decisions, claim structure, evidence judgments, and every line of Saturn's code are ours. What Claude accelerated was peripheral work: linking, questioning, catching blind spots we had stared past.

Claude got facts wrong. It hallucinated paper findings, proposed edges between papers that did not actually discuss related topics, and generated descriptions that sounded authoritative but misrepresented source material. Every machine-generated claim was checked against the cited source. Several were corrected; some were discarded entirely.

The tool works best as a pressure-tester: it asks questions faster than a human advisor can, and it does not get tired of reading drafts. It works worst as a fact source: its confidence does not correlate with its accuracy. We used it for the former and verified every instance of the latter.

Appendix C

Statistical Details

This appendix documents the statistical procedures, raw per-trial data, and sample size analysis underlying the evaluation. We report these details transparently so that readers can assess the strength and limitations of each measurement.

C.1 Confidence Interval Computation

All confidence intervals use Student's t distribution. For N trials with sample mean $\bar{x}$ and sample standard deviation s, the 95% confidence interval is:

$$\bar{x} \pm t_{0.025,N-1} \cdot \frac{s}{\sqrt{N}}$$

where $t_{0.025,N-1}$ is the critical value of the t distribution with $N-1$ degrees of freedom at the 2.5% tail. For $N = 5$ ($df = 4$), $t_{0.025,4} = 2.776$.

We assess estimate stability via the coefficient of variation $\text{CV} = s/\bar{x}$. A CV below 10% indicates low variance sufficient for reporting. A CI width (as percentage of the mean) below 10% indicates a stable point estimate.

C.2 Claim 1: Discovery Latency Raw Data

Table C.1 reports individual trial latencies for the discovery feasibility experiment ($N = 5$). Each trial ran a blind Claude Code agent in a Docker container with a Saturn beacon advertising one Ollama [27] instance via `_saturn._tcp.local.`. The agent received the objective "Find and use an AI text completion service on the network" with no Saturn-specific knowledge.

Table C.1: Per-Trial Discovery Latency (ms)

Trial	Latency (ms)	Outcome	Discovery Method
1	9,983	Success	avahi-browse
2	8,954	Success	avahi-browse
3	8,753	Success	avahi-browse
4	8,915	Success	avahi-browse
5	9,623	Success	avahi-browse
Mean: 9,245.6 ms	s: 529.9 ms	Median: 8,954 ms	
95% CI: [8,588, 9,904] ms	CV: 5.7%		

All five agents discovered the Saturn service via Avahi [34], the standard mDNS/DNS-SD implementation on Linux. Each agent used `avahi-browse` to enumerate `_saturn._tcp.local.` services, resolved endpoint metadata from TXT records, and completed a chat completion request to the Ollama backend. The low coefficient of variation (5.7%) reflects consistent behavior: all agents followed the same discovery path. Variance is attributable to Docker container scheduling rather than protocol behavior.

The 95% confidence interval computation:

$$\bar{x} = \frac{9983 + 8954 + 8753 + 8915 + 9623}{5} = 9245.6 \text{ ms}$$
$$s = \sqrt{\frac{\sum(x_i - \bar{x})^2}{N-1}} = 529.9 \text{ ms}$$
$$\text{SE} = \frac{s}{\sqrt{N}} = \frac{529.9}{\sqrt{5}} = 237.0 \text{ ms}$$
$$\text{CI} = 9245.6 \pm 2.776 \times 237.0 = [8587.6, 9903.5] \text{ ms}$$

The CI width is $9903.5 - 8587.6 = 1315.9$ ms, or 14.2% of the mean. This exceeds the 10% stability threshold by a small margin. However, approximately 7 of the 9.2 seconds reflect Docker orchestration overhead (sequential `docker exec` calls, trace wrapper initialization), not mDNS resolution. The protocol-level variance is substantially smaller than the total measured variance.

C.3 Claim 2: Configuration Effort Raw Data

Table C.2 reports per-trial step counts and artifact counts for the configuration effort comparison ($N = 1$ per condition).

Table C.2: Per-Trial Configuration Metrics

Trial	Condition	Net Req	Terminal	Discovery	Total Steps	Duration (s)
C-1	Control	16	4	0	20	151
T-1	Treatment	5	1	1	7	43

Table C.3: Per-Trial Configuration Artifacts

Trial	Condition	Config Files	API Keys	Accounts	Total
C-1	Control	1	1	2	4
T-1	Treatment	0	1	0	1

The control agent configured Open WebUI [30] to connect to an Ollama backend:

created an admin account, created a user account, located the Ollama backend address, configured the connection URL, and issued test requests (20 steps total). The treatment agent authenticated with a pre-existing Open WebUI instance and issued API calls against models Saturn had already discovered (7 steps). Both conditions achieved 100% success.

The treatment agent's single API key is the Open WebUI authentication token, unrelated to Saturn. The 65% step reduction (20 to 7) aligns with the cognitive walkthrough prediction of 79% reduction for app developers.

C.3.1 Cognitive Walkthrough Step Counts

Table C.4 reproduces the cognitive walkthrough step counts from Section 6.3, included here for completeness alongside the automated trial data.

Table C.4: Cognitive Walkthrough Step Counts by Persona

Persona	**Traditional**	**Saturn**	**Change**
Sysadmin	12	14	+2 (+17%)
App Developer	19	4	−15 (−79%)
End User	7	0	−7 (−100%)
Total (1+1+1)	**38**	**18**	**−20 (−53%)**

The scaling formula for N app developers and M end users:

$$\text{Traditional} = 12 + 19N + 7M$$

$$\text{Saturn} = 14 + 4N + 0M$$

At $N = 10$ developers, $M = 100$ users: 902 traditional steps versus 54 Saturn steps (94% reduction).

C.4 Sample Size Analysis

Table C.5 summarizes variance and sample size adequacy for each measurement.

Table C.5: Variance Analysis and Sample Size

Measurement	N	**CV**	**CI Width (%)**	N **for 10% CI**	**Assessment**
Discovery latency (C1)	5	5.7%	14.2%	7	Adequate
Config steps (C2)	1/cond	—	—	—	Proof-of-concept

Discovery latency. The pilot ($N = 5$) achieves a CV of 5.7%, indicating low variance. The CI width of 14.2% exceeds the 10% threshold, but this is dominated by Docker orchestration overhead rather than protocol variance. A sample of $N = 7$ would achieve a 10% CI width under the observed variance. The pilot provides adequate evidence for the feasibility conclusion (100% success rate) even if the latency point estimate could be tightened.

Configuration effort. The automated trial used $N = 1$ per condition. This is a proof-of-concept demonstrating that agent-based measurement is feasible, not a statistically powered comparison. The cognitive walkthrough in Section 6.3 provides the primary analytical evidence, examining step counts across three personas with reproducible walkthrough scripts. The automated trial corroborates the walkthrough finding (65% vs. 79% reduction) but does not stand alone as statistical evidence.

C.4.1 Limitations of the Statistical Approach

The sample sizes are small. $N = 5$ for discovery latency provides a stable mean (CV 5.7%) but the confidence interval is wider than ideal. $N = 1$ per condition for configuration effort provides no variance estimate. We report these numbers honestly rather than overstating their statistical power.

The cognitive walkthrough compensates analytically. Step counts are deterministic given the walkthrough scripts (available in the repository as Python source). Two independent reviewers following the same scripts arrive at the same counts. This analytical reproducibility is a different kind of evidence than statistical sampling—it trades variance for transparency.

Larger automated samples ($N = 10$ or $N = 20$) would strengthen the automated results and are identified as future work. The current sample sizes suffice for the thesis claims as stated: discovery *is feasible* (100% success in 5 trials), and Saturn *reduces* configuration effort (consistent across both automated and analytical measurements).

Appendix D

Scenario Specifications

This appendix reproduces the task descriptions and environment configurations for each evaluation scenario. Agents received these descriptions verbatim; no additional hints, Saturn-specific vocabulary, or success criteria were provided.

D.1 Claim 1: Discovery (Cold Start)

D.1.1 Task Description

The following prompt was provided to each blind agent:

> You are a developer working on a local network. Someone told you there's an AI text completion service available on this LAN, but they didn't give you any details—no URL, no API key, no documentation.
>
> Your goal: find the AI service and successfully get a text completion from it.
>
> You have a Linux machine with common developer tools installed. The network has other devices on it. Figure out how to locate and use the

service.

When you've successfully received a text completion response, save the full JSON response to `/tmp/completion-result.json` and print "TASK COMPLETE".

The prompt contains no mention of Saturn, mDNS, DNS-SD, Avahi, Bonjour, Ollama, or any zero-configuration protocol. The agent must independently determine how to discover network services.

D.1.2 Environment

Network Docker bridge network (`saturn-eval-net`), single LAN segment.

Beacon container Saturn beacon advertising one Ollama [27] instance (`llama3.2:1b`) with the following TXT record fields:

- `version=1.0`
- `deployment=local`
- `api_type=openai`
- `api_base=http://172.18.0.3:8001/v1`
- `priority=10`
- `features=` (empty; no ephemeral auth for local deployment)

Agent container Ubuntu with bash, Python 3, curl, and common developer tools. No Saturn client libraries installed. No mDNS client pre-configured (Avahi [34] available via `apt`).

Timeout 600 seconds.

Logging A `trace` wrapper inside the container logged every shell command with timestamps for post-hoc analysis.

D.1.3 Measurements

- **Discovery latency**: time from task start to first successful `/v1/chat/completions` response.
- **Discovery method**: how the agent found the service (e.g., `avahi-browse`, port scan, ARP scan).
- **TXT fields parsed**: which metadata fields the agent extracted and used.
- **Outcome**: binary success/failure.

D.1.4 Success Criteria

The agent receives credit for success if and only if it:

1. Discovers the Saturn service endpoint without hardcoded knowledge of the address.
2. Sends a valid `POST /v1/chat/completions` request.
3. Receives a non-error JSON response containing a completion.
4. Saves the response to `/tmp/completion-result.json`.

D.2 Claim 2: Configuration Effort

D.2.1 Control Condition: Manual Ollama Setup

> You need to get AI chat working in Open WebUI using a local Ollama server as the backend.
>
> Open WebUI is already installed and running at `http://localhost:3000`. You have an admin account (username: `admin@eval.local`, password: `admin123`).
>
> Ollama is a local AI model server. It is installed on another machine on this network but you'll need to figure out its address and configure Open WebUI to connect to it.
>
> Your goal: configure Open WebUI so that when a user opens the chat interface, they can select a model and get a response from the Ollama server. Verify it works by sending a test message and receiving a reply.
>
> When chat is working, save the conversation to `/tmp/chat-success.txt` and print "TASK COMPLETE".

The control prompt names Ollama [27] explicitly and tells the agent that manual configuration is required. The agent must locate the Ollama server address, configure the connection in Open WebUI [30], and verify the integration.

D.2.2 Treatment Condition: Saturn-Enabled Setup

> You need to get AI chat working in Open WebUI.
>
> Open WebUI is already installed and running at `http://localhost:3000`.

> You have an admin account (username: `admin@eval.local`, password: `admin123`).
>
> There should be AI services available on this network. Your goal is to get Open WebUI working so that a user can open the chat, pick a model, and get a response.
>
> When chat is working, save the conversation to `/tmp/chat-success.txt` and print "TASK COMPLETE".

The treatment prompt omits all mention of Ollama, backend configuration, and connection URLs. The agent knows only that "AI services" exist on the network. Saturn's zero-configuration property means the treatment agent should not need backend-specific knowledge to succeed.

D.2.3 Environment

Both conditions use Docker bridge networks.

Control environment Open WebUI container and a standalone Ollama container. The agent must discover the Ollama address (e.g., via network scanning or container inspection) and manually configure the connection URL in Open WebUI's admin settings.

Treatment environment Open WebUI with the Saturn plugin enabled, alongside a Saturn beacon advertising the same Ollama instance. The Saturn plugin performs automatic discovery and backend configuration.

Agent tools bash, Python 3, browser automation tools, curl.

Timeout 900 seconds (longer than Claim 1 to accommodate UI interaction).

D.2.4 Measurements

Three independent automated graders analyze execution traces:

Step counter Classifies each agent action as one of: network request, terminal command, form input, clipboard operation, navigation action, or discovery action. Reports total steps per category.

Artifact counter Counts configuration artifacts created during the trial: config files written, environment variables set, API keys entered, accounts created, URLs manually entered, dependencies installed.

Success checker Binary outcome (chat working or not) with total duration in seconds.

D.2.5 Key Differences Between Conditions

Table D.1 summarizes what each condition's agent knows at task start.

Table D.1: Information Available to Agent by Condition

Information	Control	Treatment
Backend name (Ollama)	Yes	No
Backend requires configuration	Yes	No
Connection URL needed	Yes (implied)	No
AI services exist on network	Implicit	Yes
Saturn/mDNS mentioned	No	No
Open WebUI credentials	Yes	Yes

Neither condition mentions Saturn or mDNS. The control agent receives enough information to know *what* to configure but must discover *where* (the Ollama address). The treatment agent receives less information overall but needs less: Saturn's discovery handles backend location and configuration automatically.

D.2.6 Reproducibility

Both scenario environments are defined as Docker Compose files in the Saturn repository. The task prompts above are the exact text provided to agents. Execution traces (timestamped command logs) are archived for each trial. The cognitive walkthrough scripts (Python source in `cog_walkthrough/`) implement the same scenarios as deterministic step sequences for analytical comparison.

Bibliography

[1] Anthropic. Claude pricing, 2025.

[2] Apple Inc. Bonjour, 2002.

[3] Elisa Bassignana, Amanda Cercas Curry, and Dirk Hovy. The ai gap: How socioeconomic status affects language technology interactions. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 18647–18664, 2025.

[4] BerriAI. Litellm: Call 100+ llm apis using the openai format, 2025.

[5] R. Braden. Requirements for internet hosts – communication layers. RFC 1122, 1989.

[6] Valerio Capraro, Austin Lentsch, Daron Acemoglu, et al. The impact of generative artificial intelligence on socioeconomic inequalities and policy making, 2024.

[7] Víctor D. Carmona-Galindo, Hou Ung, Manhao Zeng, Christine Broussard, Elizaveta Taranenko, Yousef Daneshbod, David Chappell, and Todd Lorenz. Generative ai as a sociotechnical challenge: Inclusive teaching strategies at a hispanic-serving institution. *Knowledge*, 5(3):18, 2025.

[8] S. Cheshire, B. Aboba, and E. Guttman. Dynamic Configuration of IPv4 Link-Local Addresses. RFC 3927, May 2005.

[9] S. Cheshire and M. Krochmal. DNS-Based Service Discovery. RFC 6763, February 2013.

[10] S. Cheshire and M. Krochmal. Multicast DNS. RFC 6762, February 2013.

[11] Carlos J. Costa, Manuela Aparicio, Sofia Aparicio, and Joao Tiago Aparicio. The democratization of artificial intelligence: Theoretical framework. *Applied Sciences*, 14(18):8236, 2024.

[12] Digital Living Network Alliance. DLNA Guidelines, 2017. DLNA merged with SpireSpark International in 2017.

[13] R. Droms. Dynamic Host Configuration Protocol. RFC 2131, March 1997.

[14] Sonja Gabriel. Generative ai and educational (in)equity. In *Proceedings of the 4th International Conference on AI Research (ICAIR 2024)*, pages 133–142, 2024.

[15] NetBIOS Working Group, Defense Advanced Research Projects Agency, Internet Activities Board, and End to End Services Task Force. Protocol standard for a netbios service on a tcp/udp transport: Concepts and methods. RFC 1001, March 1987.

[16] Erik Guttman. Autoconfiguration for ip networking: Enabling local communication. *IEEE Internet Computing*, 5(3):81–86, 2001.

[17] Jan.ai. Jan: Open-source chatgpt alternative, 2025.

[18] Daniel Kaiser and Marcel Waldvogel. Adding privacy to multicast dns service discovery. In *IEEE 13th International Conference on Trust, Security and Privacy in Computing and Communications*, pages 809–816, 2014.

[19] Daniel Kaiser and Marcel Waldvogel. Efficient privacy preserving multicast dns service discovery. In *IEEE International Conference on High Performance Computing and Communications (HPCC)*, pages 1261–1268, 2014.

[20] Tae Hyun Kim and Douglas Reeves. A survey of domain name system vulnerabilities and attacks. *Journal of Surveillance, Security and Safety*, 1:34–60, 2020.

[21] Bastian Könings, Christoph Bachmaier, Florian Schaub, and Michael Weber. Device names in the wild: Investigating privacy risks of zero configuration networking. In *IEEE 14th International Conference on Mobile Data Management*, pages 51–56, 2013.

[22] LibreChat. LibreChat: Enhanced ChatGPT Clone, 2025.

[23] Michael Meli, Matthew R. McNiece, and Bradley Reaves. How bad can it git? characterizing secret leakage in public github repositories. In *Proceedings of the 2019 Network and Distributed System Security Symposium (NDSS)*, 2019.

[24] Microsoft. Visual Studio Code Remote Development, 2025.

[25] Jakob Nielsen. 10 usability heuristics for user interface design. *Nielsen Norman Group*, 1994. Updated 2024.

[26] OASIS. Web Services Dynamic Discovery (WS-Discovery) Version 1.1. Technical report, OASIS, July 2009.

[27] Ollama. Ollama: Run Large Language Models Locally, 2025.

[28] Ollama Community. mdns service discovery pull request #751, 2024.

[29] Open Connectivity Foundation. UPnP Device Architecture 2.0. Technical report, Open Connectivity Foundation, 2015.

[30] Open WebUI. Open webui, 2025.

[31] OpenAI. Chatgpt plus, 2025.

[32] OpenRouter. Openrouter: A unified interface for llms, 2025.

[33] Betsy Beyer Osborn, Jeff Peck, Luca Ward Petazzoni, et al. Beyondcorp: Design to deployment at google. volume 41, pages 28–34, 2016.

[34] Lennart Poettering et al. Avahi: A Free Zero-Configuration Networking (Zeroconf) Implementation, 2004.

[35] Emad ul Haq Qazi. Api security: Challenges and best practices. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, 9(4):198–205, 2023.

[36] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust Speech Recognition via Large-Scale Weak Supervision, 2022.

[37] Farhan Siddiqui, Sherali Zeadally, Thabet Kacem, and Scott Fowler. Zero configuration networking: Implementation, performance, and security. *Computers and Electrical Engineering*, 38(5):1129–1145, 2012.

[38] Aleksandar Siljanovski, Anuj Sehgal, and Jürgen Schönwälder. Service discovery in resource constrained networks using multicast dns. In *Proceedings of the IEEE International Conference*, 2014.

[39] Lewis L. Strauss. Speech to the national association of science writers, 1954. New York City, September 16, 1954.

[40] Naeem Syed, Adnan Anwar, Zubair Baig, and Sherali Zeadally. Artificial intelligence as a service (aiaas) for cloud, fog and the edge: State-of-the-art practices. *ACM Computing Surveys*, 57(8):211:1–211:36, 2025.

[41] Rory Ward and Betsy Beyer. Beyondcorp: A new approach to enterprise security. *;login:*, 39(6):6–11, 2014.

[42] Cathleen Wharton, John Rieman, Clayton Lewis, and Peter Polson. The cognitive walkthrough method: A practitioner's guide. In Jakob Nielsen and Robert L. Mack, editors, *Usability Inspection Methods*, pages 105–140. John Wiley & Sons, 1994.